
Projet de session

C64 | Objets connectés

Table des matières

Introduction générale	5
Découpage de l'application	5
Découpage du développement	5
Livrables et calendrier proposé	6
Objectifs	7
Considérations générales	7
Étape préliminaire Modules disponibles	11
Présentation générale	11
Détails techniques et stratégies d'appropriation	11
Diagrammes de classes	12
Partie I Première couche d'abstraction Application de dispositifs de suivi	14
Présentation générale	14
Module tracking_device	14
Diagrammes de classes	16
Présentation détaillée des classes	18
Développement attendu	18
Livrables attendus	18
Stratégies le développement	19
Partie II Deuxième couche d'abstraction Machine d'états	20
Présentation générale	20
Réflexion sur l'effort d'abstraction	20
Module state_machine_device	21
Diagrammes de classes	22
Diagramme de séquence	25
Développement attendu	26
Livrables attendus	26
Partie III Extensions standards de la bibliothèque de machine d'états	27
Présentation générale	27
La couche des conditions	27
La couche des actions	27
La couche de surveillance (<i>monitored</i>)	27
Indépendance et séparation	28
Modules condition et state_device_utilities	28

Diagrammes de classes	29
Développement attendu.....	32
Livrables attendus	32
Partie IV Extension spécialisée Première liaison au projet	34
Présentation générale	34
Dispositifs intermittents	34
Généralisation du concept.....	34
Conception proposée	35
Module blinker_device	35
Diagramme de classes	36
Diagramme d'états-transitions.....	38
Développement attendu.....	38
Livrables attendus	38
Partie V Développement de la couche applicative	40
Présentation générale	40
Diagramme d'états-transitions	41
Détails sur chacun des états de l'application principale	43
Importance de la structure flexible associée aux tâches	45
Conception à réaliser	45
Diagrammes UML	46
Tâche de contrôle manuel	47
Détails sur l'état manual_control	47
Diagramme états-transitions.....	48
Détails sur les états de la tâche manual_control	48
Développement attendu.....	49
Livrables attendus	50
Partie VI Développement de tâches supplémentaires	51
Présentation générale	51
Sujet de la tâche	51
Contraintes techniques.....	51
Qualité de la tâche	52
Finalisation du projet	53
Rapport	53
Stratégie d'évaluation	54

Remise	55
Annexe 1 – UML	56
Rappel sur la notation UML et directives supplémentaires générales	56

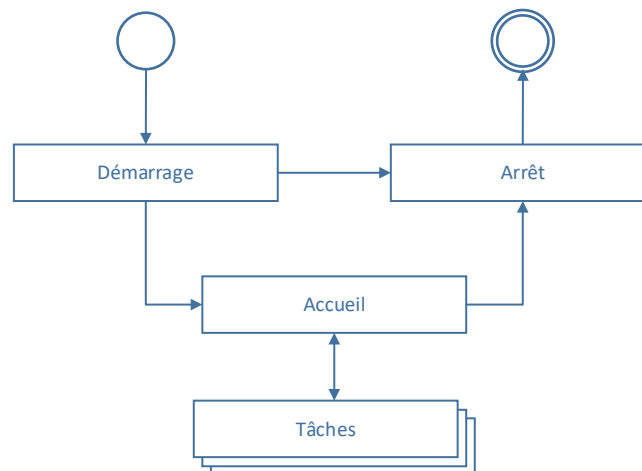
Introduction générale

Le projet de session consiste à développer un logiciel permettant l'exécution de diverses tâches basées sur le robot GoPiGo3.

Découpage de l'application

L'application principale est décomposée de quatre sections :

1. Démarrage : Amorce du logiciel, instanciation et validation de l'intégrité du robot.
2. Arrêt : S'assure de mettre le robot dans une situation sécuritaire.
3. Accueil : Le robot est en attente et l'utilisateur est invité à lancer une tâche.
4. Tâche : Exécution d'une tâche sélectionnée parmi celles disponibles.



Sections principales de l'application à développer

Découpage du développement

D'un point de vue technique, le projet est découpé en trois parties principales :

- A. Développement d'une bibliothèque implémentant une machine d'états générique et indépendante de ce projet. Cette bibliothèque est elle-même appuyée par d'autres couches d'abstraction :
 - a. application en flux tiré
 - b. machine d'états
 - c. outils génériques de la machine d'états (conditions, actions et surveillance)
 - d. outils spécialisés pour dispositifs de suivi intermittents
- B. Développement de l'infrastructure logicielle principale de l'application, un logiciel flexible et modulaire.
- C. Développement de tâches variées dont chacune joue un rôle spécifique.

Quant à lui, le développement se déroulera en six étapes distinctes :

1. **A1** - Développement de la première couche d'abstraction générique :
la bibliothèque **TrackingDevice** :
 - a. Classes utilitaires et dépendances.
 - b. Noyau et moteur fondamental de la bibliothèque d'application en flux tiré.
2. **A2** - Développement de la deuxième couche d'abstraction générique :
la bibliothèque **StateMachineDevice**
 - a. Noyau et moteur fondamental de la machine d'états.
3. **A3** - Développement d'extensions standards de la machine d'état
 - a. Mise en place de la généralisation du concept de condition dans les transitions.
 - b. Extension facilitant une intégration de dispositifs en flux poussé.
 - c. Extension d'un mécanisme de surveillance permettant l'automatisation de certaines tâches récurrentes.
4. **A4** - Exploitation de la bibliothèque par le développement d'extensions spécialisées mais génériques
 - a. Gestion des dispositifs à deux états (ouvert/fermé).
5. **B** - Développement de la couche applicative par la mise en place de l'application principale.
 - a. Représentation du robot.
 - b. L'application principale.
 - c. La première tâche : le contrôle manuel du robot.
6. **C** - Ajout de tâches supplémentaires :
 - a. **C1** - Tâche personnalisée 1 – tâche **ManualControl**
 - b. **C2** - Tâche personnalisée 2
 - c. ...
 - d. **Cn** - Tâche personnalisée n

Livrables et calendrier proposé

Malgré le découpage technique présenté, deux livrables sont à considérer pour ce projet :

1. Développement de la bibliothèque `state_machine_device` (étapes 1 à 3).
2. Développement de l'application finale (inclus les deux dernières étapes 4 et 6).

Le projet est réparti sur ces 9 semaines de développement :

Semaine 1. Prise en main :

- i. présentation générale
- ii. formation des équipes
- iii. lecture de ce document
- iv. appropriation de l'infrastructure donnée

Semaine 2. Étape 1

Semaine 3. Étape 2

Semaine 4. Étape 3

Semaine 5. Consolidation du premier livrable et premier examen.

Semaine 6. Étape 4

Semaine 7. Étape 5

Semaine 8. Étape 6

Semaine 9. Consolidation du deuxième livrable et dernier examen.

Objectifs

Ce projet vise à donner aux étudiants finissants une opportunité de consolider plusieurs notions de leur formation collégiale. L'objectif étant d'exploiter certains concepts essentiels et fondamentaux du développement logiciel afin d'en garantir une compréhension satisfaisante. Pour y arriver, les considérations pédagogiques suivantes sont mises de l'avant :

- usage intensif du paradigme orienté objet
- capacité d'interprétation de diagrammes UML (diagrammes de classe, de paquetage, de séquence et d'états)
- découpage du projet en plusieurs couches d'abstraction
- séparation des développements liés aux éléments génériques et de l'application elle-même
- exploitation de plusieurs patrons de conception
- mise en œuvre de pratiques valorisées dans l'industrie (typage et validation statique du code, `docstring`, test unitaires, séparation des dépendances, ...)

Finalement, ce projet est exigeant et demande un certain niveau de maturité lié au développement logiciel. Les étudiants doivent s'impliquer et garantir leur compréhension du projet par un effort de travail soutenu. Le projet n'est pas difficile en soit si on considère uniquement le code lui-même. Toutefois, le projet est très exigeant par sa forme et son niveau d'abstraction.

Il est attendu que **tous** les étudiants posent des questions afin de décortiquer et comprendre l'architecture proposée. Les discussions parallèles avec l'enseignant sont **essentiels** pour réussir le projet. Ce sont ces échanges qui vous donneront la perspective suffisante pour apprécier entièrement le projet en cours de réalisation et vous permettre de mettre en lien tous ses sous-aspects.

Finalement, ce projet requiert une charge de travail substantielle en classe et à l'extérieur des cours (rappelez-vous la pondération 2-2-3). Il est aussi important que le travail d'équipe soit partagé afin de garantir le succès de tous les étudiants aux évaluations.

Considérations générales

1. Votre dépôt **GIT** doit **obligatoirement** être privé et votre enseignant doit y être invité.
2. Le travail doit être réalisé en équipe :
 - a. La taille des équipes est définie par l'enseignant.
 - b. Ce n'est pas un travail où certains étudiants doivent se laisser porter par les autres ou, à l'inverse, qu'un étudiant se sentent investi d'une mission sacrée et fasse tout le travail au détriment de l'équipe.
 - c. Il est important de trouver un équilibre de travail. De former des sous équipes et, autant en classe qu'à la maison, déterminer les éléments qui doivent être réalisés à deux et les éléments qui seront développés seuls.
 - d. La pratique de la programmation en binôme (parfois en trinôme) est considérée comme fondamentale dans ce projet. Les techniques de gestion de projet vues dans les cours de génie logiciel sont recommandées.
3. Le niveau de ce projet est :
 - a. Techniquement facile en termes d'écriture du code.

- b. Conceptuellement difficile à cause du niveau d'abstraction à plusieurs niveaux. Il requiert un effort soutenu pour comprendre chacun des niveaux d'abstraction demandé. Lire la partie 5 peut aider à mieux comprendre pourquoi le projet est construit de cette façon.
 - c. La charge de travail est calibrée en fonction de la taille des équipes, des heures en classe et des heures à faire à la maison. On tient aussi compte des compétences que vous avez acquises pendant votre formation mais aussi des attentes liées au marché du travail.
 - d. Le niveau du projet valorise plusieurs connaissances que vous avez acquises pendant votre DEC à travers plusieurs cours et divers enseignant.e.s. De plus, le même projet permet de faire une mise en commun de ces apprentissages tout en explorant quelques concepts plus avancés basés sur vos acquis. Autrement dit, c'est un projet intégrateur valorisant la transition entre le DEC et votre vie professionnelle future. C'est une transition entre deux états!
4. Consultation avec l'enseignant :
- a. La valeur de ce projet réside dans les discussions avec l'enseignant.
 - b. Autrement dit, il est impossible de réaliser ce travail sans poser des questions et d'en tirer tous les apprentissages.
 - c. Il est attendu que vous soyez curieux et que vous posiez des questions sur la conception présente. Autant pour sa compréhension que pour les choix qui ont été faits (compromis) que pour l'identification des abstractions exploitées.
 - d. Pour favoriser au maximum l'apprentissage d'équipe, il est **obligatoire** que chaque étudiant pose des questions. Ainsi, à chaque question, vous devez déterminer qui posera la question dans l'équipe et changer à chaque fois de façon à ce que chaque étudiant pose une question.
 - e. Les évaluations tiennent pour acquis que vous poserez des questions pour garantir votre compréhension du projet. Certaines questions des évaluations tiennent pour acquis que certaines questions évidentes aient été discutées avec l'enseignant.
 - f. Au final, il ne suffit pas de faire fonctionner le code, mais de comprendre son essence.
5. On vous demande de développer une éthique de travail saine et de mettre en pratique les recommandations de l'industrie :
- a. Le développement incrémental est une excellente façon d'optimiser son temps et sa compréhension.
 - b. Prendre le temps d'expliquer à ses pairs ou de se faire expliquer par ses pairs le travail en cours ou le travail réalisé est une excellente opportunité d'améliorer ses compétences de communication et de vulgarisation. Dites-vous que plus vous expliquerez un concept, plus vous articulerez votre raisonnement et mieux vous serez capable de l'expliquer et de le justifier. Ces acquis se reflèteront grandement dans votre vie professionnelle et particulièrement en entrevue. Rappelez-vous que si vous n'êtes pas capable d'expliquer un sujet simplement, c'est que vous ne le comprenez pas suffisamment.
 - c. Mettre en pratique les recommandations de travail suivantes :
 - i. **UMUD** (yoU Must Use the Debugger)
 - ii. **CALTAL** (Code A Little, Test A Little)
 - iii. **DRY** (Don't Repeat Yourself)
 - iv. **OFOT** (One Function, One Task)
 - v. **MAP** (Modularity Above Performance, if the latter is not required or necessary)
6. D'un point de vue technique, portez une attention aux points suivants. Vous trouverez tous les détails dans les sections spécifiques des livrables de chacune des étapes du projet :
- a. La langue d'usage est :

- i. En anglais pour tout le code (le nommage des types et des variables).
 - ii. En français pour la `docstring`.
 - iii. En anglais ou en français pour les commentaires. Toutefois, restez consistant dans une seule de ces langues au choix de l'équipe.
- b. Le typage :
 - i. Vous devez effectuer toutes les annotations de type pour les 5 premières parties du projet.
 - ii. Vous devez effectuer la validation statique du code pour les 4 premières parties du projet (utilisation de `mypy`).
- c. Validation des intrants :
 - i. Vous devez valider le domaine de toutes les entrées de vos méthodes (en type et valeur) pour les 4 premières parties du projet (la bibliothèque).
 - ii. Si une entrée est invalide, vous devez lever l'exception appropriée.
- d. `docstring`, `doctests`, `UnitTest` et documentation par un site web :
 - i. Pour chacun des 2 livrables, chaque étudiant de l'équipe doit sélectionner l'une des classes les plus pertinentes du travail réalisé. Ensuite, chaque étudiant doit :
 1. Produire la `docstring` complète de la classe. Vous devez mettre l'emphasis sur la qualité de la documentation et de la pertinence de la présentation des interfaces de programmation publiques.
 2. Produire les `doctests` pour cette documentation. Les tests ne doivent pas être exhaustifs mais plutôt servir à présenter quelques exemples simples d'usage.
 3. Vous devez produire un site web documentaire des classes documentées à l'aide de `pdoc3`.
 4. Vous devez produire les tests unitaires exhaustifs de la classe en utilisant la bibliothèque de `Python UnitTest`. Vous devez créer un fichier nommé `test_xyz.py` où `xyz` est le nom du module à tester.
- e. Les commentaires :
 - i. Le niveau des commentaires doit être minimum mais présents là où nécessaires. Autrement dit :
 - ii. Votre code devrait être auto-documenté directement par les noms des types, des variables et par les annotations de type.
 - iii. L'interface de programmation publique devrait être documenté par les `docstrings`.
 - iv. Les commentaires devraient uniquement être présents dans la logique du code pour les parties sensibles, complexes ou nécessitant des explications pertinentes. Jamais pour écrire en français ou en anglais le code écrit en `Python`. En industrie, il n'est jamais accepté de laisser des commentaires creux, ces commentaires qui n'ajoutent rien.
- f. Vous trouverez dans le document de travail une structure de dossiers à respecter (**obligatoire**). À la racine de votre dossier de travail se trouve les sous dossiers suivants :
 - i. `dev` : Tous les fichiers de développement (code et test) doivent se retrouver directement dans ce dossier. Il ne doit pas exister de sous dossier. Vous y retrouverez autant les fichiers `Python` donnés que ceux qui sont à produire.

- ii. **doc** : Tous les documents de documentation générés se retrouvent dans ce dossier. Le sous-dossier **pdoc3** ne doit pas être modifié (d'ailleurs, vous y retrouverez un sous-dossier nommé **template** nécessaire à **pdoc3**).
- iii. Le dossier **misc** vous appartient et permet de stocker toutes autres informations que vous jugez nécessaires. Il peut aussi rester vide.
- iv. Le dossier **eval** contient la grille d'auto-évaluation **Excel** que vous devez remplir deux fois (pour chacune des deux remises).
- v. À la racine se trouve un **readme.md** qui doit être rempli par l'équipe. N'oubliez pas d'indiquer tous les membres de l'équipe aussitôt le projet déposé sur **Github**. On s'attend à un document non exhaustif mais tout de même pertinent.

Présentation générale

Pour débiter le projet, on met à votre disposition trois modules facilitant certains aspects du développement. Même si chacun d'entre eux est très simple, il importe d'en prendre connaissance afin de les maîtriser. Vous devez vous approprier leur contenu comme s'il était réalisé par un membre de l'équipe. Ils sont sujet à évaluation et correspondent au niveau de code à réaliser.

Détails techniques et stratégies d'appropriation

Voici un sommaire de ces modules :

- module `type_utilities` : alias de type communs et récurrents.
- module `elapsed_timer` : contient la classe, `ElapsedTimer`, dont le rôle est de faciliter l'évaluation de l'écoulement du temps.
- module `base_component` : contient la classe, `BaseComponent`, l'une des classes les plus importantes de tout le projet car elle forme plusieurs classes fondamentales des bibliothèques mises en place.

Pour ces trois classes, vous avez les documents suivants :

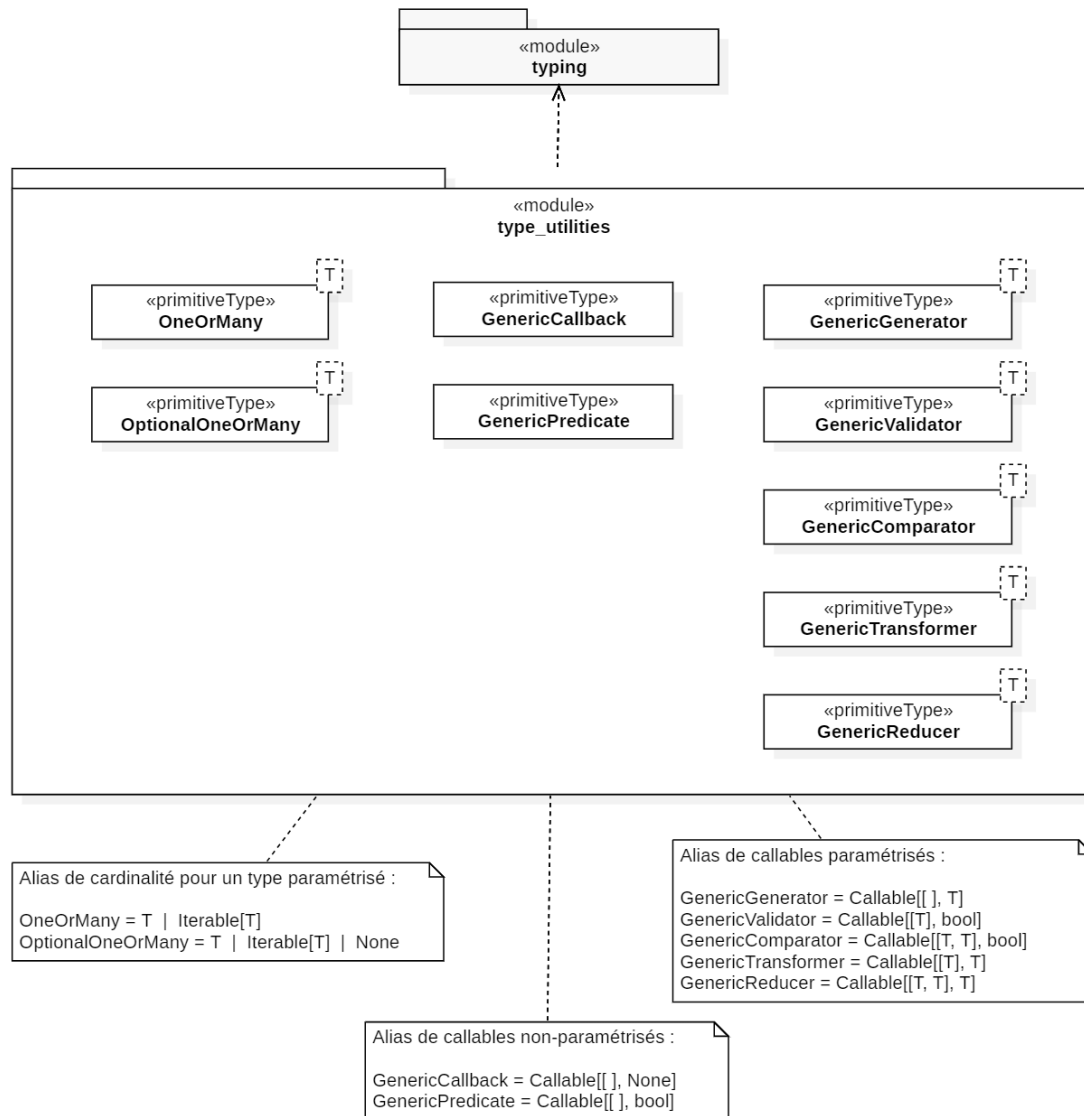
- dans le dossier `dev` :
 - pour chaque module, un fichier `Python` dans lesquels vous trouverez le code, la `doctring` et la `doctest` :
 - `type_utilities.py`
 - `elapsed_timer.py`
 - `base_component.py`
 - Un fichier de test unitaire de chaque module :
 - `test_type_utilities.py`
 - `test_elapsed_timer.py`
 - `test_base_component.py`
- dans le dossier `doc` :
 - une page web documentaire pour chaque module :
 - `type_utilities.html`
 - `elapsed_timer.html`
 - `base_component.html`
 - Un sous dossier propre à l'exécution de `pdoc3` (ne pas modifier sans l'accord de l'enseignant)

Sachez aussi que tous ces modules ont été validées par `mypy` et que la documentation a été générée par `pdoc3`.

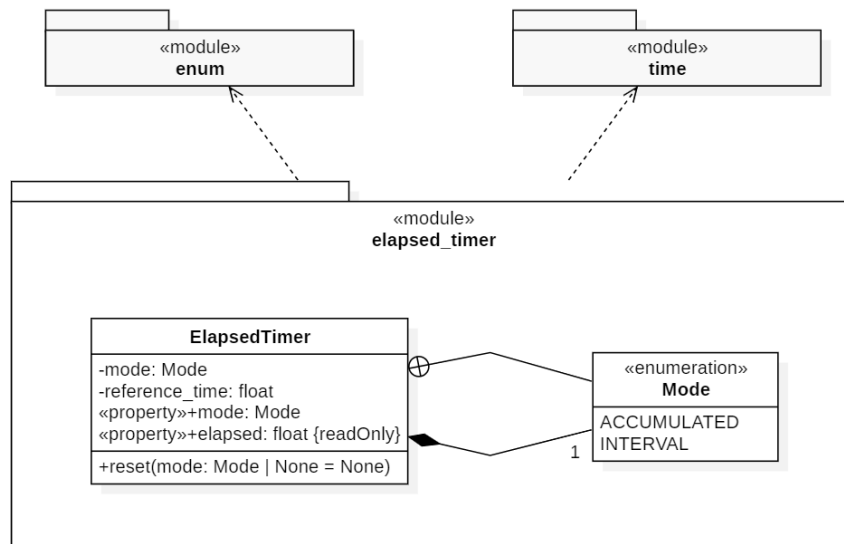
Pour l'appropriation de ces modules, on vous encourage à faire les analyses et les apprentissages par la pratique du *pair-programming*. Ensuite, valider votre compréhension en équipe. Soyez attentif dans ces lectures car plusieurs concepts et détails techniques sont expliqués. En même temps, faites la lecture du code, du site web documentaire et des diagrammes de classes **UML** donnés dans ce document. Cette approche vous aidera à mieux faire les liens entre les diverses représentations des mêmes constituants logiciels : analyse, production et documentation.

Le contenu de tous ces fichiers est sujet aux évaluations (code, documentation et test).

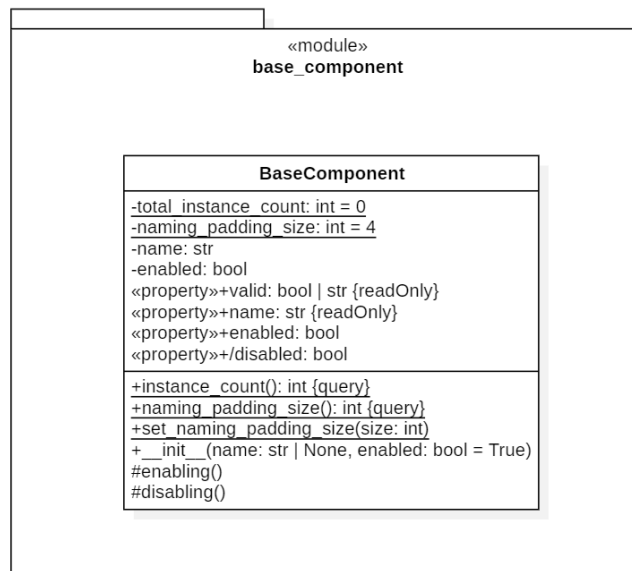
Diagrammes de classes



Module `type_utilities`



Module `elapsed_timer`



Module `base_component`

Partie I Première couche d'abstraction | Application de dispositifs de suivi

Présentation générale

Le projet débute par le développement d'une bibliothèque facilitant la création d'applications génériques basées sur des composants internes opérant en flux tiré.

Le concept de flux tiré (*pull-based programming*) repose sur la capacité qu'un constituant puisse se mettre à jour lui-même en allant chercher lui-même les informations dont il dépend. Dans ce document on fait référence à ces composants par le terme **dispositif de suivi**.

Le concept d'une application en flux tiré repose sur l'implémentation d'une boucle principale exécutée de manière continue de façon à garantir la mise à jour de tous les constituants internes. Chaque composant a la responsabilité de maintenir sa cohérence et son intégrité. Cette approche diffère fondamentalement du modèle en flux poussé (*push-based programming* se trouvant à la base de la programmation événementielle *event-driven*), où les actions sont déclenchées de manière réactive en réponse à des événements spécifiques. Le flux tiré permet un contrôle centralisé de bas niveau sur l'ensemble des opérations, garantissant une synchronisation stricte et uniforme de tous les composants. En revanche, le modèle en flux poussé, bien que plus réactif, présente des défis en matière de prévisibilité quant au séquençement des actions. De plus, il est intéressant de noter que certaines implémentations de l'approche événementielle sont basées sur le modèle en flux tiré.

La conception d'une bibliothèque générique pour une application en flux tiré est une tâche intrinsèquement complexe et de grande envergure, notamment en raison des défis liés à la synchronisation des constituants et à la gestion des dépendances entre ceux-ci en lien avec les contraintes du système d'exploitation. Toutefois, cette complexité peut être grandement atténuée en identifiant les caractéristiques fondamentales et en les structurant de manière flexible. En simplifiant le niveau technique et en basant le développement sur le paradigme de programmation orientée objets, il est possible de créer une gamme d'outils offrant un bon niveau d'abstraction avec une infrastructure relativement simple.

L'approche proposée maximise la réutilisabilité du code, assurant ainsi une modularité efficace qui facilite l'adaptation aux besoins futurs spécifiques et variés. Par exemple, la modularité permet de réutiliser des modules existants pour intégrer rapidement de nouvelles fonctionnalités, telles que l'ajout d'un nouveau type de dispositif. Cela réduit le temps de développement et garantit une compatibilité fluide avec les autres composants du système.

Finalement, il est essentiel de comprendre que cette approche a été simplifiée afin de rendre le projet accessible dans le cadre de la formation collégiale. Ainsi, plusieurs considérations de développement logiciel ont été volontairement mises de côté : les événements de bas niveau comme les interruptions, le développement asynchrone, l'architecture multithread, et autres. Ces choix ont été faits dans un contexte pédagogique mais, dans un contexte de développement professionnel, ces autres aspects devraient être intégrés.

Module `tracking_device`

Le module `tracking_device` contient la bibliothèque du même nom et vise à permettre la création d'applications basées sur la gestion de dispositifs de suivi utilisant une approche en flux tiré.

La bibliothèque repose sur les concepts suivants :

- L'application est composée de :
 - Une collection d'instances de dispositif de suivi représentant les constituants de l'application : chaque objet joue un rôle spécifique dans le processus global, rendant le système plus modulaire et gérable.
 - Une boucle principale à exécution rapide : cette boucle est essentielle pour l'actualisation régulière de chaque constituant, assurant une mise à jour synchronisée.
 - Un mécanisme d'évaluation du temps écoulé entre chaque itération : cela permet de maintenir une synchronisation uniforme des constituants lors des mises à jour.
- Les dispositifs de suivi, quant à eux, possèdent les caractéristiques suivantes :
 - Un nom unique, permettant la distinction de chaque dispositif.
 - Une capacité de vérification de conformité, facilitant l'identification précoce de problèmes potentiels.
 - Un état actif ou inactif, permettant de contrôler la mise à jour de manière sélective.
 - Une hiérarchie de sous-dispositifs internes, permettant d'étendre les fonctionnalités de manière flexible.
 - Une fonction de mise à jour, permettant la synchronisation de l'état du dispositif avec la boucle principale. Cette fonction est cruciale pour maintenir la cohérence des informations entre tous les composants et assurer que chaque dispositif soit en phase avec les changements apportés par le système.

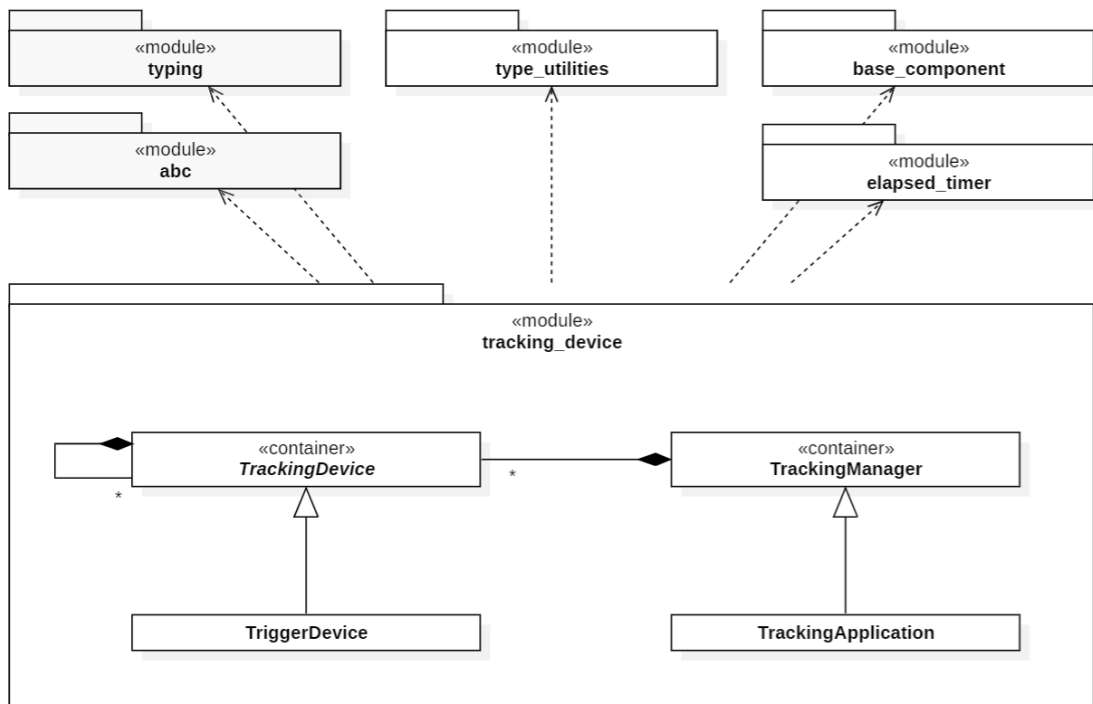
La bibliothèque est composée de trois classes fondamentales génériques :

- **TrackingDevice** : dispositif de suivi générique destiné au polymorphisme
- **TrackingManager** : gestionnaire de plusieurs dispositifs
- **TrackingApplication** : l'application gérant les dispositifs de suivi

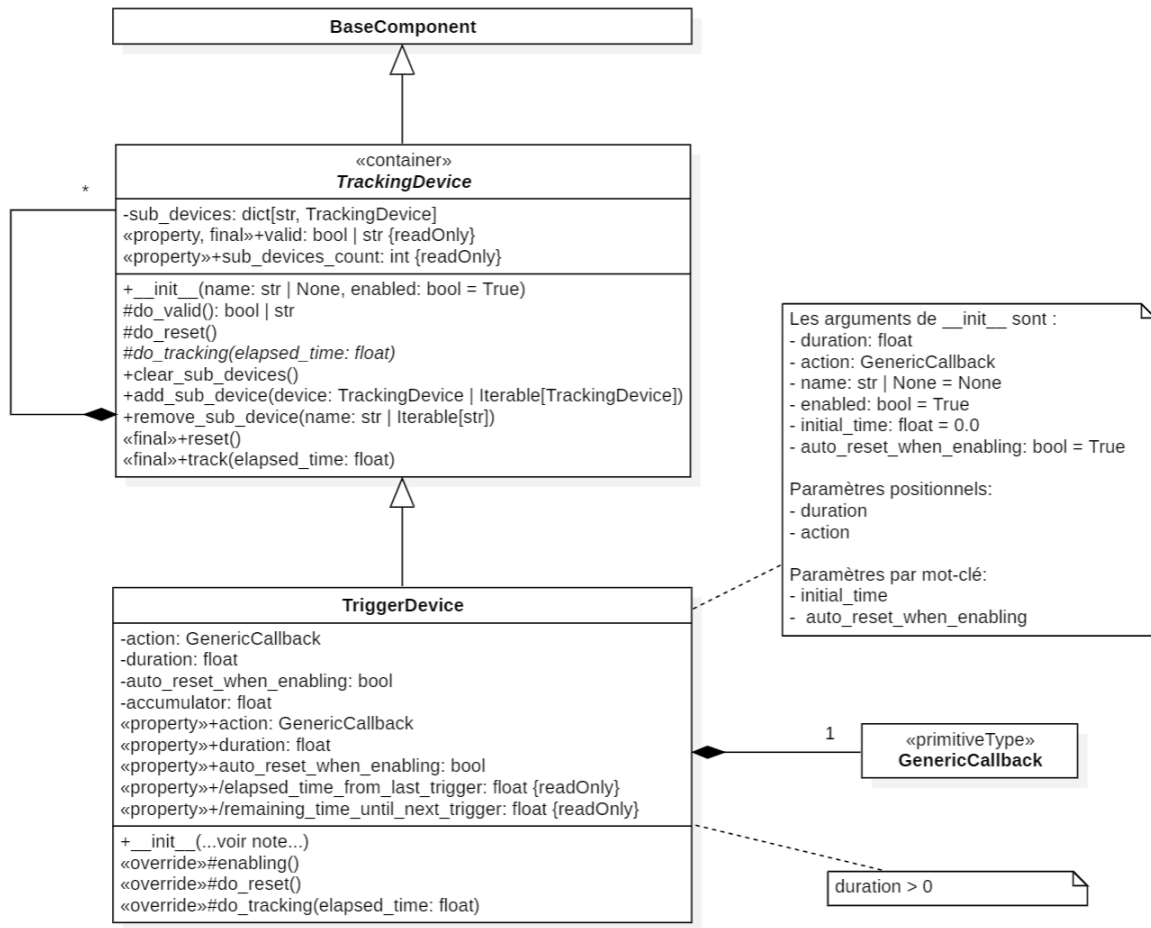
Et d'une classe utilitaire :

- **TriggerDevice** : un premier exemple de dispositif de suivi

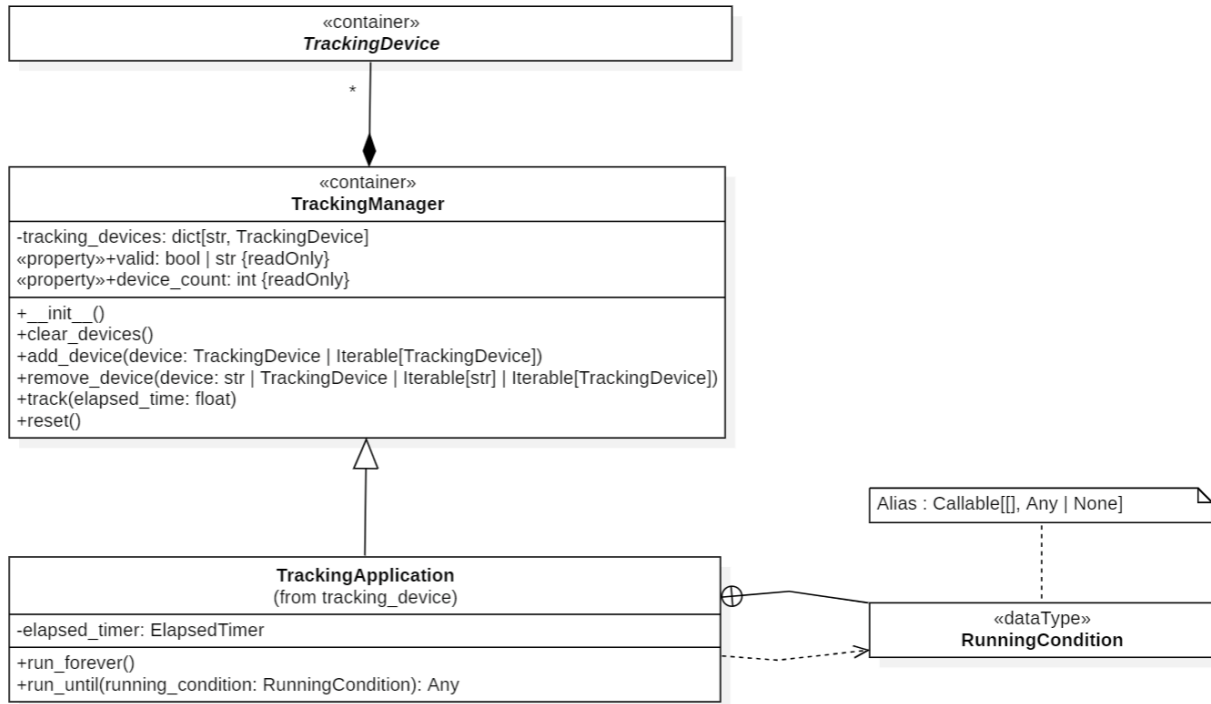
Diagrammes de classes



Vue d'ensemble du module `tracking_device`



Classes TrackingDevice et TrackingTrigger



Classes **TrackingManager** et **TrackingApplication**

Présentation détaillée des classes

Un site web documentaire technique sur ce module a été généré avec **pdoc3**. Ce dernier est mis à votre disposition pour donner tous les détails sur ce diagramme **UML**. La lecture de ces documents est essentielle.

Développement attendu

Vous devez développer entièrement la bibliothèque **tracking_device** et suivre rigoureusement la conception proposée. Sauf avis contraire par l'enseignant, vous ne devez pas modifier la conception existante.

Livrables attendus

- Les 4 classes doivent être réalisées dans un seul fichier nommé **tracking_device.py**.
- Vous devez tester votre bibliothèque :
 - Vous devez créer un micro-projet fictif pour tester la bibliothèque.
 - Vous devez obligatoirement réaliser un autre **device** de votre cru. Ce dispositif peut être très simple.
 - Suggestion possible, faites une classe qui affiche à l'écran un texte dans la console. La fréquence d'affichage n'est pas liée au temps mais plutôt au nombre de fois que la fonction **track** est appelée (par exemple, à chaque 100 appels). Ainsi, la classe possède deux paramètres, le texte à afficher et le nombre d'appels provoquant l'affichage.
 - Les classes **TriggerDevice**, **TrackingApplication** ainsi que votre dispositif de suivi personnalisé doivent être utilisés explicitement comme base de test.
 - Ce test doit être dans un fichier nommé **project_tracking_device.py**.

- Sachez que pour cette partie du projet, il n'est pas attendu que toutes les fonctionnalités soient testées. Toutefois, sachez que c'est vraiment à votre avantage de le faire. On s'attend que l'entête décrive l'objectif du test et quels résultats doivent être observés (en `docstring`). Vous n'avez pas à :

Stratégies le développement

1. Assurez-vous de comprendre les concepts fondamentaux et les objectifs de cette bibliothèque.
2. Faites une lecture sommaire des diagrammes de classes afin d'établir des liens conceptuels et avoir une vue d'ensemble de la conception **UML** proposée.
3. Assurez-vous d'avoir une compréhension minimum des dépendances requises pour faire ce module (la classe **BaseComponent** par exemple).
4. Faites une lecture approfondie des diagrammes de classes afin de comprendre les nuances, subtilités et choix de conception qui ont été faits. Il faut comprendre que l'architecture donnée est issue d'une suite de choix tentant de maximiser les possibilités tout en minimisant la complexité et la charge de travail. Comme dans tout projet, un ensemble de compromis a été réalisé.
5. Développer les classes selon leur ordre de dépendance (les diagrammes **UML** présentent clairement cette dépendance) :
 - a. **TrackingDevice**
 - b. **TrackingManager**
 - c. **TrackingApplication**
 - d. **TriggerDevice** (peut être développée en parallèle avec les tâches *b* et *c* puisque la dépendance n'est que pour la tâche *a*).
6. Pendant le développement, vous devriez construire progressivement les contenus suivants :
 - a. Réalisation des `docstring`. Vous avez déjà le contenu des textes via les pages `html` documentaires.
 - b. Votre bibliothèque doit être validée par `mypy`.

Présentation générale

La notion de machine d'état représente un concept fondamental en informatique, utilisé pour modéliser le comportement d'un système en fonction d'un ensemble d'états distincts et des transitions qui les relient. Un système peut être défini comme étant dans un état spécifique à un instant donné, et peut évoluer d'un état à un autre en réponse à certains événements ou conditions. Cette formalisation permet de structurer des systèmes complexes en décomposant leur comportement en plusieurs étapes gérables, tout en garantissant une cohérence interne. Les machines d'état trouvent leurs racines dans divers domaines, allant de la théorie des automates à l'électronique numérique, et sont désormais omniprésentes dans les systèmes logiciels, en particulier pour modéliser des flux de contrôle, des protocoles ou des processus.

Une machine d'état se compose essentiellement d'états, de transitions et d'événements. Les états représentent les différentes situations ou configurations dans lesquelles le système peut se trouver. Les transitions décrivent les passages d'un état à un autre, généralement en réponse à des événements spécifiques. Les événements, quant à eux, constituent les déclencheurs externes ou internes, capables de provoquer une modification de l'état du système. Ce cadre théorique s'applique aussi bien à des scénarios simples qu'à des systèmes très complexes, tels que les systèmes embarqués, les protocoles de communication ou encore les interfaces utilisateur, où chaque interaction avec l'utilisateur déclenche une série de transitions soigneusement définies.

L'importance de la machine d'état réside dans sa capacité à structurer le comportement d'un système de manière claire et déterministe. En effet, elle permet de formaliser les règles qui régissent les transitions entre différents états, ce qui facilite non seulement la conception, mais également la maintenance du système. Ce type de modélisation est particulièrement précieux dans des contextes où la prévisibilité et la rigueur sont primordiales, comme dans le domaine des systèmes critiques ou des applications où la moindre erreur de transition pourrait entraîner des conséquences majeures.

Dans une perspective de développement logiciel, l'utilisation d'une machine d'état permet d'abstraire des processus complexes en composants réutilisables et modulaires, offrant ainsi une meilleure maintenabilité et évolutivité. Développer un outil réutilisable et générique se fait assez naturellement avec le paradigme orienté objet. Encore une fois, cette approche maximise la réutilisabilité du code et permet une adaptation aisée à des besoins futurs spécifiques, comme l'ajout de nouveaux états ou transitions dans un système évolutif, sans compromettre la cohérence du modèle.

Même s'il est possible de développer une machine d'états avec une approche en flux tiré ou en flux poussé, l'infrastructure logicielle proposée est basée sur la bibliothèque `tracking_device`. Par conséquent, il faudra créer un dispositif de suivi conceptuellement abstrait, une machine d'états générique!

Réflexion sur l'effort d'abstraction

Le développement logiciel de cette machine d'états générique représente la partie la plus abstraite du projet. Il est crucial de prendre du recul et de s'éloigner des spécificités du projet en cours pour concevoir ou comprendre la conception d'un outil véritablement générique et réutilisable. Pour se faire, il peut être utile de se concentrer sur les concepts fondamentaux plutôt que sur les détails d'implémentation. L'objectif est de créer une bibliothèque indépendante, non pas comme un simple détail du projet actuel, mais comme un composant logiciel robuste et polyvalent, pouvant être intégré dans de multiples projets, y compris celui-ci.

La perspective est que cette bibliothèque soit réutilisable autant pour divers besoins au sein du même projet, que pour des projets indépendants, et qu'elle supporte l'évolution future des projets qui l'exploiteront.

L'effort nécessaire pour s'approprié de tels réflexes de développement n'est pas facile et demande du temps et de l'expérience. Toutefois, le gain est immense et, pour une petite bibliothèque comme celle-ci, immédiat. Autrement dit, l'effort d'abstraction n'est pas seulement profitable à long terme pour des projets futurs mais immédiatement pour ce projet. Il permet de construire une entité réutilisable et d'encapsuler des concepts spécifiques, facilitant le développement, le déverminage et la maintenance.

Pour favoriser cette prise de recul essentielle, plusieurs approches peuvent être envisagées.

- L'étude de cas concrets d'utilisation de machines d'états dans des contextes variés, allant des automates industriels aux jeux vidéo et aux sites web, offrira une perspective plus large et inspirante.
- La réalisation de diagrammes et de schémas facilitera la visualisation et la compréhension des interactions entre les différents états et transitions. Cet effort aidera à identifier les éléments clés de la machine d'états générique.
- L'organisation de séances de *brainstorming* et d'échanges avec d'autres développeurs (vos collègues mais aussi l'enseignant) permettra de confronter les idées, de partager les connaissances et de stimuler la créativité.
- Finalement, la consultation de la littérature scientifique et technique sur les machines d'états permettra de se familiariser avec les différentes implémentations et les meilleures pratiques.

En somme, il s'agit de s'immerger dans le concept de machine d'états, d'en explorer les multiples facettes et de s'en imprégner afin de concevoir une bibliothèque logicielle simplifiant le projet en ajoutant de la valeur au produit et à l'entreprise. Il est important de réaliser que l'effort intellectuel nécessaire pour s'approprier de tels réflexes de développement est quelque chose qui se pratique progressivement dans votre carrière mais qu'il faut mettre de l'avant!

Module `state_machine_device`

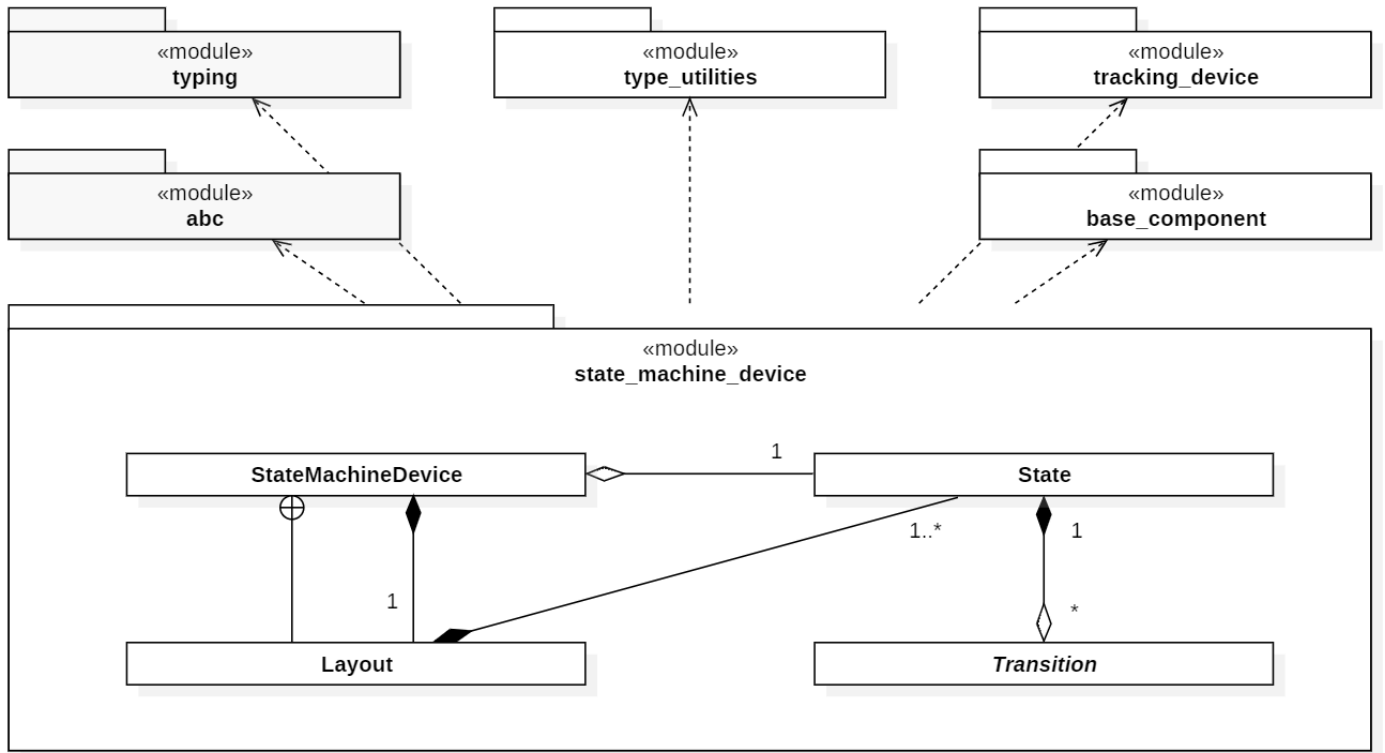
Le module `state_machine_device` implémente la machine d'états dans sa forme la plus pure et la plus générique. Autrement dit, ce module comporte l'essence même de la machine d'état sans artifice ou utilitaires. Cette approche garanti une séparation essentielle entre l'engin de la machine d'état et n'importe quel usage qu'il sera possible d'en faire. Ce sera le rôle d'autres modules que d'ajouter des outils pour aider à son usage (les prochaines étapes).

On retrouve dans ce modules les trois abstractions du concept de machine d'états. Ces constituants essentiels prennent forme, encore une fois, via le paradigme orienté objets qui est mis de l'avant :

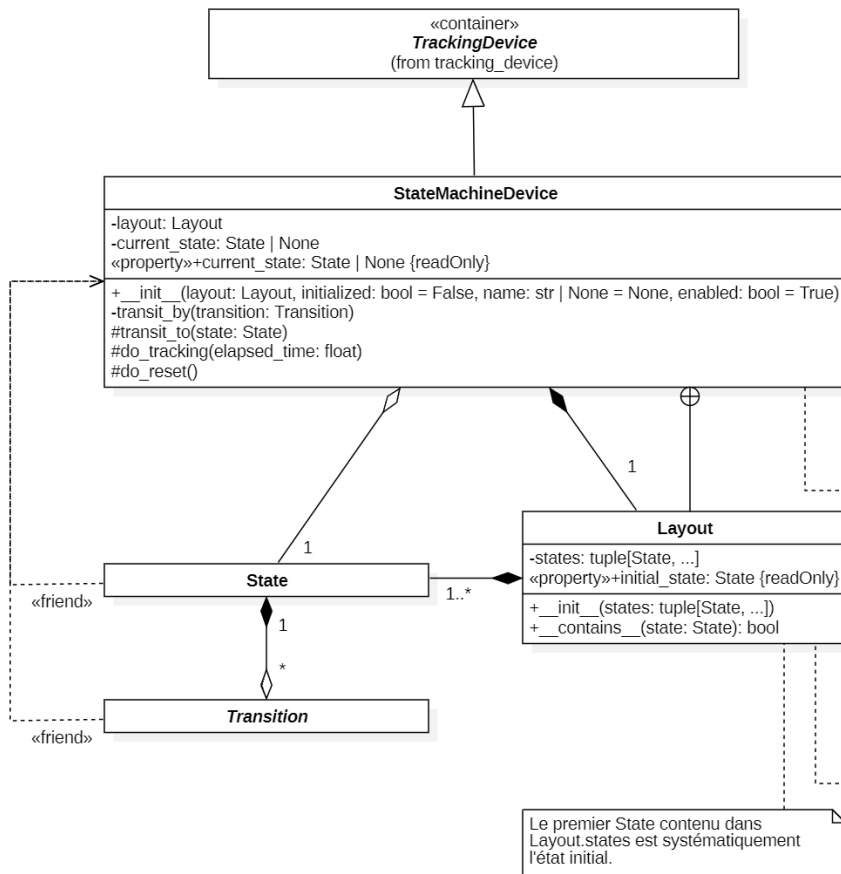
- La classe `State` représentant un état.
- La classe `Transition` représentant une transition.
- La classe `StateMachineDevice` représentant l'engin de la machine d'état, qui est aussi un dispositif de suivi. Cette classe possède une sous-classe utilitaire nommée `Layout` représentant le schéma de tous les états de la machine d'état instanciée.

Un diagramme de séquence **UML** est introduit pour illustrer le fonctionnement interne de la machine d'états. La fonction `StateMachineDevice.track` joue un rôle central dans l'architecture du projet, en assurant la gestion des transitions et l'actualisation de l'état courant. En outre, elle est responsable de l'exécution séquentielle des opérations et du déclenchement automatique des événements associés.

Diagrammes de classes



Vue d'ensemble du module `state_machine_device`



La conception proposée est originale et nécessite une certaine attention à sa compréhension.

Malgré le fait que la classe StateMachineDevice ne soit pas abstraite, elle est destinée à être héritée. Autrement dit, il devrait être impossible de pouvoir l'instancier directement.

Pour créer une machine d'état spécifique à une situation, l'idée consiste à forcer l'héritage en créant une nouvelle classe. Dans le constructeur de cette classe enfant, on crée le Layout en lui donnant tous les états et les transitions nécessaires. Une contrainte technique importante est nécessaire pour cette approche, il faut que l'appel du constructeur parent soit réalisé APRES la création de l'objet Layout.

Cette approche favorise une encapsulation serrée et rend opaque le Layout de l'extérieur. En contrepartie, ce design oblige l'application de l'héritage, ce qui pourrait nuire à de petits projets.

Finalement, il faut être conscient qu'un tel design est possible en Python mais plusieurs langages ne le permettent pas. Par exemple, Java, C# et C++ n'ont aucune flexibilité sur l'ordre d'appel du constructeur parent. Dans ces langages, les constructeurs parents sont appelés systématiquement avant le constructeur enfant. Rendrant cette approche impossible (le C++ peut le contourner avec le CRTP qui est une notion avancée et déroutante!).

La variable initialized indique que la machine d'états est initialisée ou non dès sa construction. Suivant la valeur donnée, l'état courant doit :

- si vrai, pointer sur l'état initial si vrai
- si faux, être à None

L'objectif ici est de ne pas forcer l'entrée dans l'état initial dès l'instanciation de la machine d'état. Il faut comprendre que l'activation de l'état initial engendre un appel indirect mais systématique à do_entering_action.

Finalement, la fonction reset garantie en tout temps cette initialisation.

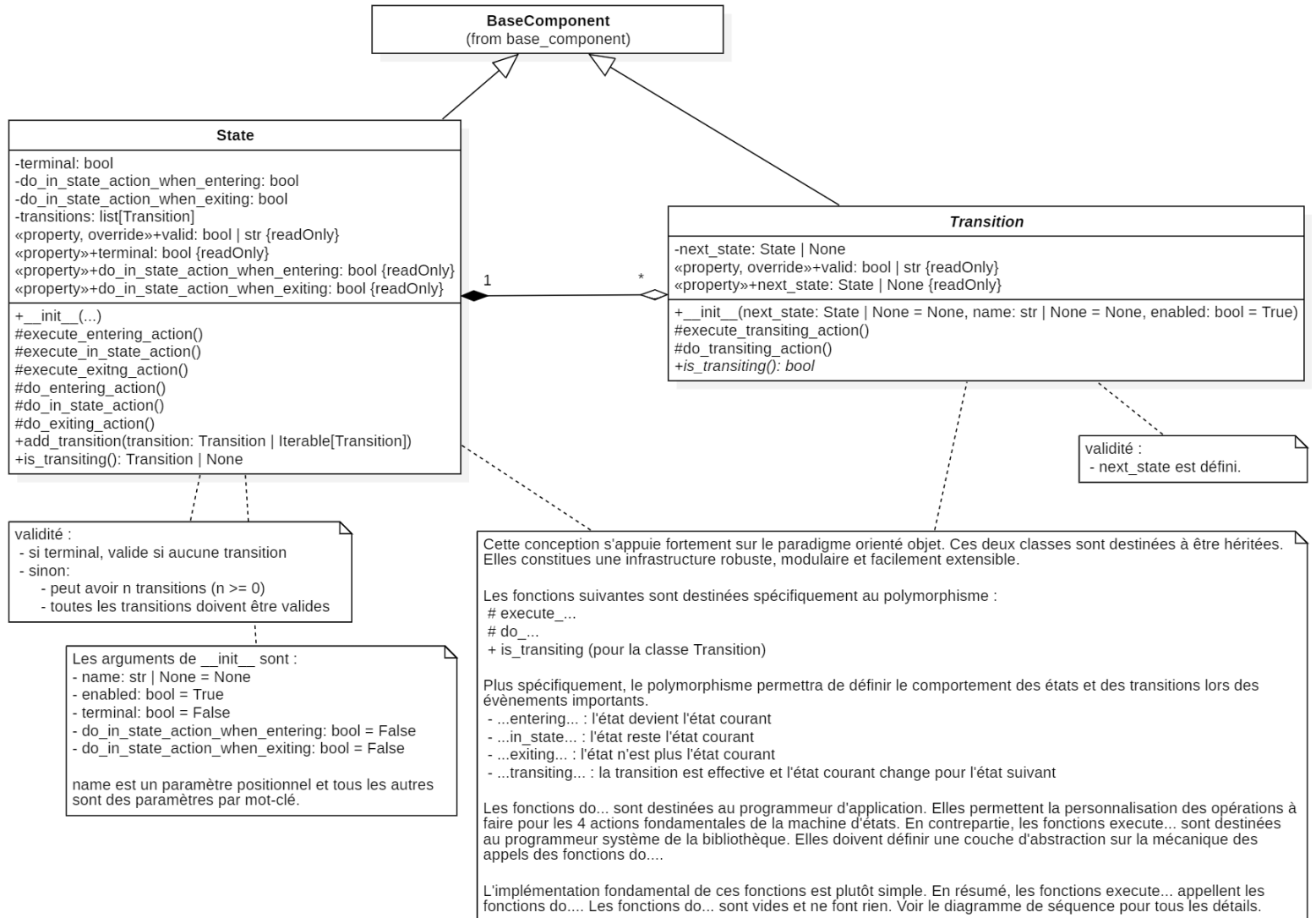
Le __init__ doit faire les validations suivantes :

- valider le typage des instances dans states
- n > 0 (le nombre de state)
- tous les states sont valides

Si une validation échoue, une exception est levée.

Par conséquent, un objet Layout est toujours valide! Ainsi, un objet StateMachineDevice est lui aussi toujours valide!

Classes StateMachineDevice et Layout



Classes State et Transition

Diagramme de séquence

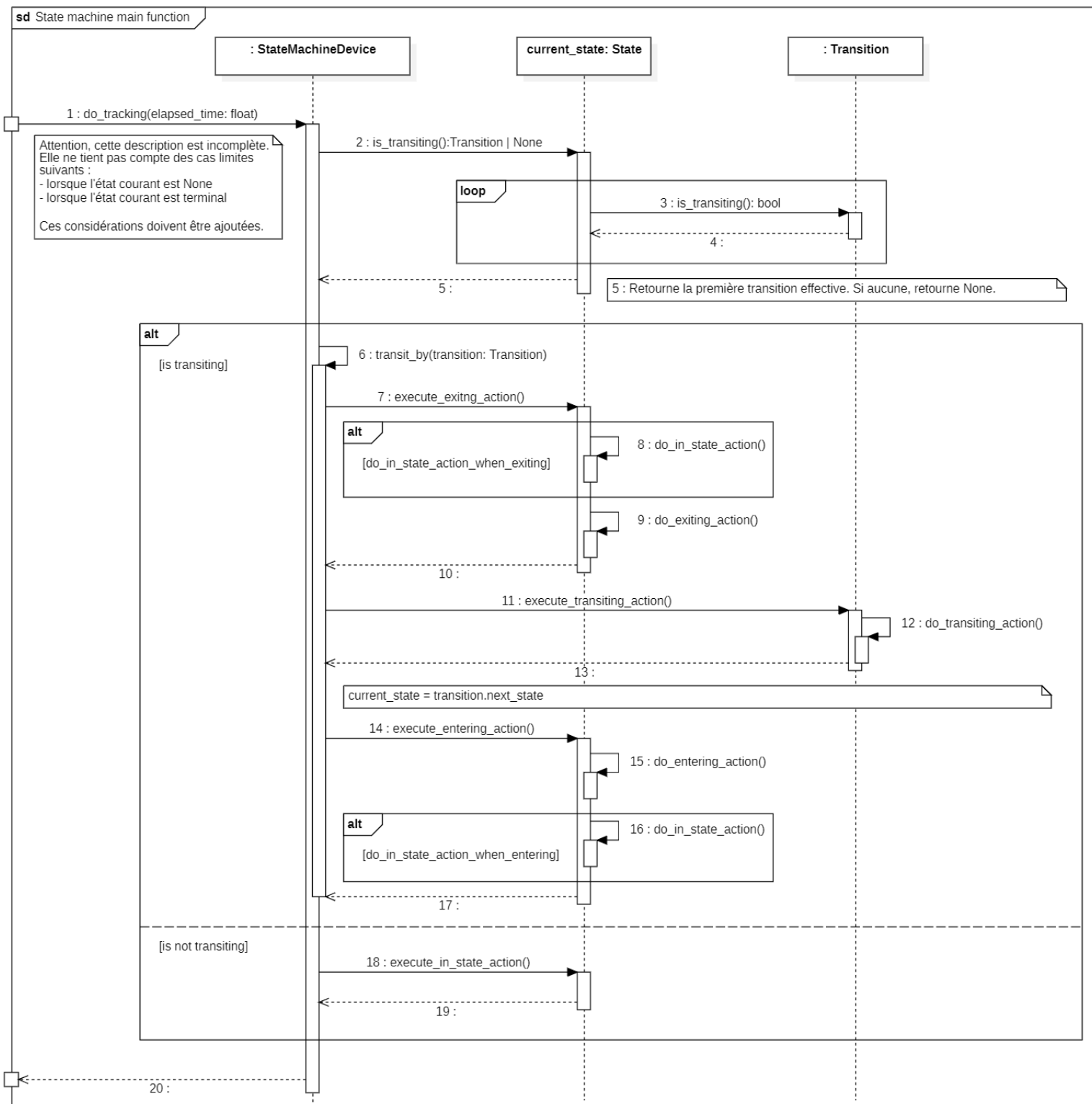


Diagramme de séquence de la fonction fondamentale `StateMachineState.track` (incluant les détails internes)

Développement attendu

Vous devez développer entièrement la bibliothèque `state_machine_device` et suivre rigoureusement la conception proposée. Sauf avis contraire par l'enseignant, vous ne devez pas modifier la conception existante.

Livrables attendus

- Les 3 classes doivent être réalisées dans un seul fichier nommé `state_machine_device.py`.
- Aucun test supplémentaire n'est obligatoire. Vous aurez à le faire ultérieurement. Néanmoins, il est vraiment à votre avantage de le faire.

Partie III Extensions standards de la bibliothèque de machine d'états

Présentation générale

Cette bibliothèque a pour vocation de faciliter la gestion des systèmes à états en fournissant une architecture à trois couches : condition, action, et surveillance (*monitored*). Elle est conçue pour gérer des systèmes où des décisions doivent être prises de manière flexible et automatisée, en réponse à des changements d'état, à des événements ou à des délais. Son objectif est de rendre les machines à états plus réactives et intelligentes, tout en offrant une interactivité de haut niveau.

On peut résumer son rôle par trois couches conceptuelles ayant chacun un rôle précis.

La couche des conditions

Au cœur de la bibliothèque se trouve un mécanisme permettant de déléguer la gestion des conditions à d'autres composants. Plutôt que d'enfermer la logique conditionnelle au sein d'une classe, celle-ci est déléguée via un pont à des classes composées, créant une structure modulaire. Cette délégation permet de séparer clairement la définition des conditions de leur implémentation, rendant ainsi le système plus adaptable et extensible. On peut ainsi spécifier des règles conditionnelles qui varient en fonction du contexte sans modifier la structure de la machine à états elle-même.

La couche des actions

Ici, la bibliothèque adopte un paradigme proche de la programmation fonctionnelle, où les transitions entre les états ou les changements internes à un état sont traités sous forme de flux poussés. Cela permet de réagir automatiquement à ces changements sans avoir à recourir uniquement à des mécanismes d'héritage pour définir des comportements spécifiques à chaque transition ou action. En effet, en plus du paradigme orienté objet, où le polymorphisme permet de spécialiser les transitions et états, cette bibliothèque introduit un second paradigme : la programmation fonctionnelle¹. Les actions peuvent ainsi être définies comme des entités indépendantes, et leur appel est automatisé par le système lors des événements appropriés. Ce double paradigme offre une flexibilité accrue, permettant de combiner l'élégance du polymorphisme avec la puissance des fonctions autonomes, qui s'exécutent au bon moment sans intervention manuelle. L'intégration de ce modèle fonctionnel permet d'éviter une complexité accrue dans la hiérarchie des classes tout en enrichissant les capacités d'automatisation et de réactivité du système.

La couche de surveillance (*monitored*)

Enfin, la bibliothèque introduit une couche de surveillance qui permet d'interagir avec le système à un niveau supérieur d'abstraction. Cette couche permet de suivre en continu l'évolution des états et des conditions, d'observer des changements subtils et d'en tenir compte dans les actions déclenchées. Grâce à cette surveillance proactive, la machine à états devient non seulement réactive mais aussi interactive, capable de s'adapter à des comportements plus complexes ou imprévus en surveillant de près son propre fonctionnement.

¹ Il ne s'agit pas ici de programmation fonctionnelle stricte, mais plutôt d'un mécanisme qui s'en inspire et s'en rapproche. L'idée repose sur la notion de fonctions autonomes déclenchées en réponse à certaines conditions, sans nécessairement suivre les principes d'immuabilité, d'effets de bord ou d'états purs propres à la programmation fonctionnelle classique. Ce concept présente également des parallèles avec le patron de conception observateur, où des opérations peuvent s'enregistrer et être appelées lors des événements appropriés. La nuance est vraiment faible si on considère la généralisation des *callable*s en *Python*. Malgré tout, l'approche utilisée ne présente pas le formalisme du patron de conception.

Cette architecture repose sur une séparation claire des responsabilités et une délégation intelligente des comportements, favorisant ainsi une grande flexibilité et une réutilisation des composants. Le système est conçu pour être extensible : les conditions, actions et mécanismes de surveillance peuvent évoluer indépendamment, permettant d'adapter la machine à états à une grande variété de situations et de besoins. Cela rend possible la construction de systèmes à états plus dynamiques, capables de s'adapter aux variations et aux événements complexes, tout en maintenant une logique claire et relativement bien structurée.

En intégrant ces trois couches, la bibliothèque permet aux développeurs de créer des systèmes sophistiqués en automatisant plus facilement certaines tâches récurrentes.

Indépendance et séparation

L'un des aspects les plus marquants de cette architecture est l'abstraction des conditions, qui s'avère totalement indépendante de cette bibliothèque. En effet, le concept de *condition* est si générique qu'il dépasse largement le cadre de la gestion des machines à états. Que l'on travaille avec des systèmes de transitions ou non, la capacité de définir et d'évaluer des conditions dans un contexte configurable dynamiquement est utile dans une grande variété de projets en informatique.

Ce détachement conceptuel indique une dépendance unidirectionnelle des rôles et souligne l'importance de créer un module distinct pour les conditions. On insiste ici sur l'importance de fournir un cadre réutilisable pour gérer des conditions dans n'importe quel type de projet. En d'autres termes, la généricité de l'abstraction des conditions appelle naturellement à sa séparation en une bibliothèque indépendante.

Il est évident que les transitions et les états dépendent des conditions pour prendre des décisions, mais la réciproque n'est pas vraie : les conditions n'ont pas besoin de la machine à états pour exister. Elles sont universelles et peuvent être appliquées à tout projet nécessitant une logique conditionnelle, qu'il s'agisse de vérifier si une certaine variable a atteint une valeur, si un certain temps s'est écoulé, ou si un ensemble de critères complexes est satisfait.

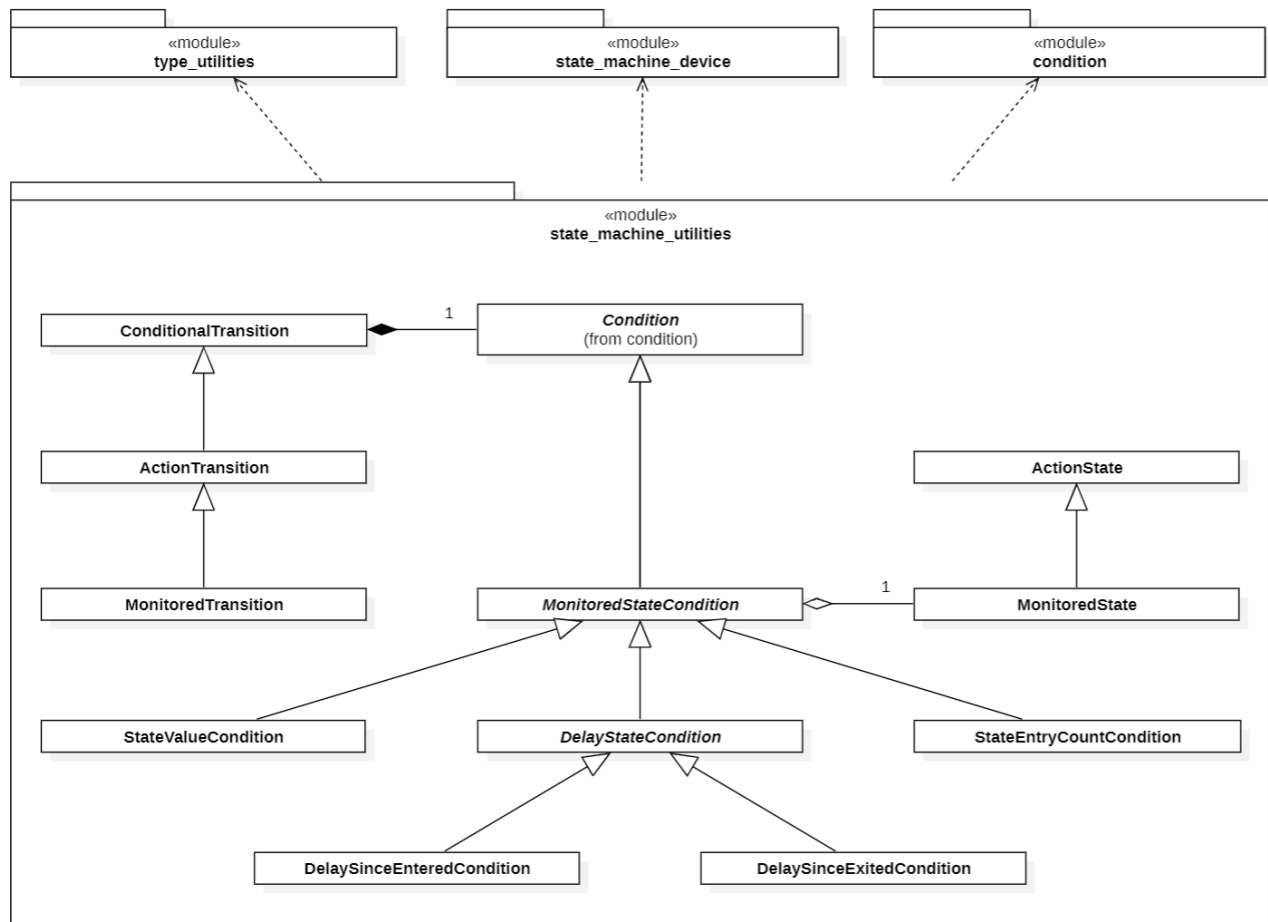
En créant une bibliothèque indépendante pour la gestion des conditions, on offre la possibilité de réutiliser ce puissant mécanisme dans une large gamme de contextes. Cela encourage également un design modulaire, où chaque composant est autonome, facilitant ainsi la maintenance et l'évolutivité. Cette approche favorise aussi un développement et une maintenance plus facile.

Finalement, cette indépendance permettra de mettre en place facilement le pont pour la délégation des conditions dans l'infrastructure déjà en place. Cet ajout précoce dans l'infrastructure permettra une adoption plus large de son usage.

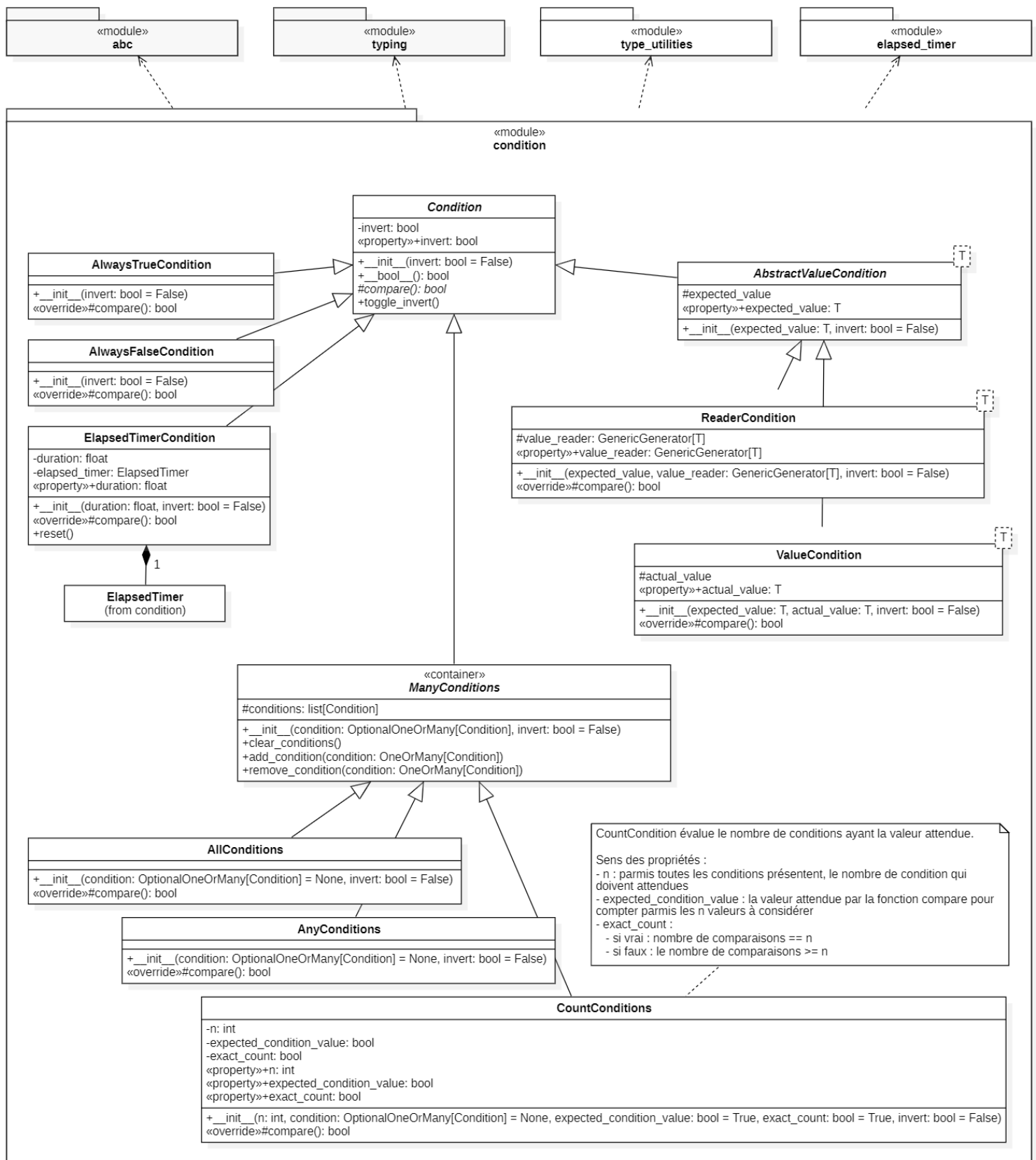
Modules `condition` et `state_device_utilities`

- Le module `condition` offre une infrastructure générique pour l'évaluation de condition flexible.
- Le module `state_device_utilities` offre les trois couches conceptuelles déjà présentées (condition, action et surveillance).

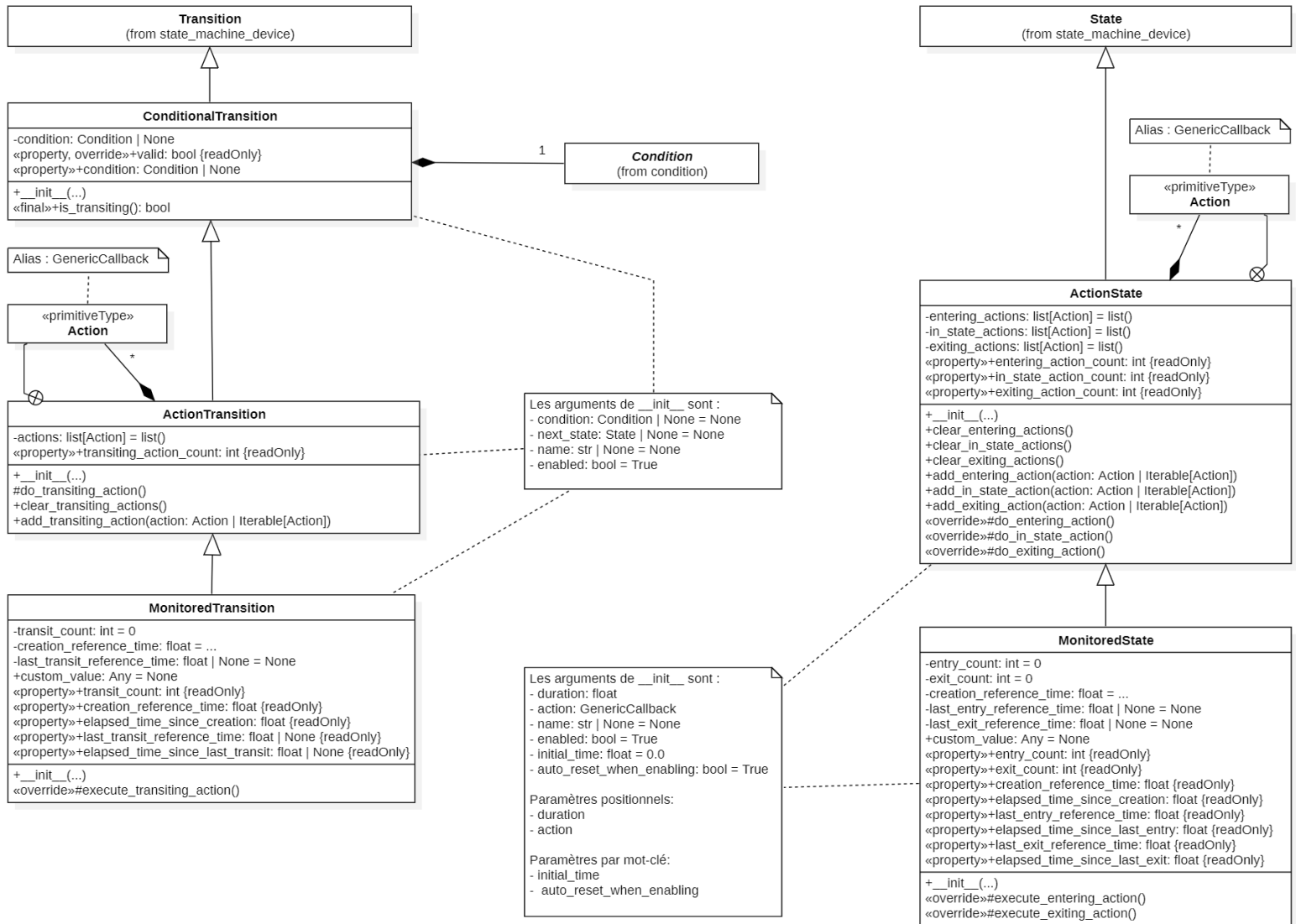
Diagrammes de classes



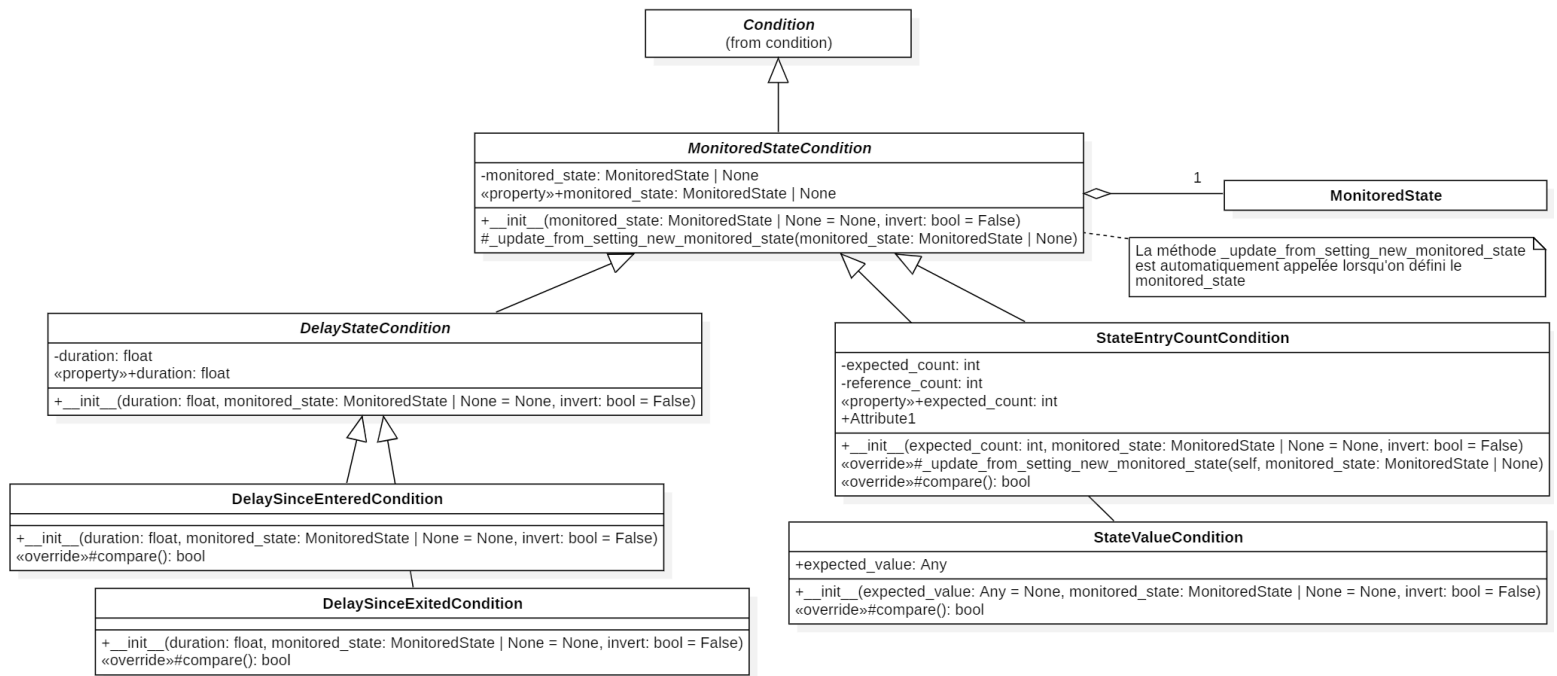
Vue d'ensemble du module `state_machine_utilities`



Module **condition**, classe **Condition** et classes génériques dérivées



Classes ConditionalTransition, ActionTransition, ActionState, MonitoredTransition et MonitoredState



Classes **MonitoredStateCondition** et classes utilitaires dérivées

Développement attendu

Vous devez développer entièrement les modules `condition` et `state_machine_utilities` en suivant rigoureusement la conception proposée. Sauf avis contraire par l'enseignant, vous ne devez pas modifier la conception existante.

Livrables attendus

- Chacun des modules doit être réalisé dans son propre fichier :
 - `condition.py`
 - `state_machine_utilities.py`

À ce stade du développement, vous avez en main une bibliothèque fonctionnelle et utilisable directement pour de petits projets bien ciblés. Autrement dit, il n'est pas systématiquement requis de mettre en pratique le polymorphisme pour créer une application. On vous demande de valider votre projet en créant une très petite application visant à tester votre code et à démontrer votre compréhension à son usage. Vous devez simuler un feu de circulation qui répond à ces requis :

- Vous avez les trois états traditionnels correspondant aux lumières rouge, vert et jaune (trois instances de la même classe). L'état initial est l'état rouge.
- Les durées entre chaque transition sont :
 - 1.0 seconde : **rouge** => **vert**
 - 0.8 seconde : **vert** => **jaune**
 - 0.2 seconde : **jaune** => **rouge**
- De plus, la machine d'états doit s'arrêter d'elle-même lorsque l'état rouge a été activé 5 fois.
- Lorsqu'un état devient effectif (l'état courant), on affiche sur la console la couleur appropriée (toujours sur la même ligne).

- Ce programme doit démontrer une bonne encapsulation.
- Le fichier doit s'appeler `traffic_light_simulation.py`.

Sachez qu'il est possible de réaliser ce petit travail avec l'approche polymorphique ou l'approche par action. Peu importe l'approche que vous prendrez, assurez-vous de savoir comment faire l'autre.

Prenez note qu'une des deux approches est meilleure que l'autre.

Présentation générale

À ce stade du développement, il est important de reconnaître et d'apprécier l'ampleur des efforts réalisés. L'infrastructure générique mise en place permet d'aborder une vaste gamme de problématiques techniques et d'application dans de nombreux domaines.

Cependant, avant d'amorcer le développement spécifique de la couche applicative du projet, on remarque que le développement d'un outil semi-spécialisé est nécessaire pour simplifier les développements à venir. Cet outil concerne les dispositifs intermittents, définis comme des éléments présentant deux états distincts : allumé ou éteint.

Dispositifs intermittents

Le robot à interfacier nécessite un contrôle non trivial de certains éléments internes. En particulier, cela inclut des composants comme les phares, les yeux, la diode électroluminescente disposée à l'arrière du robot et, éventuellement, d'autres actuateurs qui pourraient être ajoutés comme une sonnerie (*buzzer*). La difficulté provient du fait que l'interface de programmation de la bibliothèque `GoPiGo3` ne permet pas d'engager des actions récurrentes autonomes, chaque action devant être explicitement définie au temps prescrit. Par exemple, pour faire clignoter un œil à une fréquence donnée, il est nécessaire de gérer explicitement les étapes d'activation et de désactivation. Cela signifie qu'il faut attendre un délai spécifique pour activer la lumière puis attendre de nouveau pour la désactiver. Bien que cette gestion soit relativement simple dans un projet de petite envergure, elle devient rapidement problématique dans un contexte plus complexe où l'on souhaite appliquer des comportements divers à plusieurs dispositifs simultanément, tout en exécutant d'autres tâches (comme la lecture de la télécommande ou le déplacement du robot). À cela s'ajoute la difficulté de modifier certaines caractéristiques comme la couleur ou l'intensité lumineuse en fonction du contexte, ou de gérer des événements imprévisibles.

Avec un peu de recul, il apparaît clairement que la gestion d'un **dispositif intermittent** constitue une tâche lourde, qu'une classe spécialisée pourrait prendre en charge de manière efficace et réutilisable. En d'autres termes, il est pertinent de concevoir et développer une entité dont le rôle serait de gérer l'état de ces dispositifs, tout en restant simple et flexible d'usage. Par exemple, cette classe pourrait permettre de configurer un dispositif pour clignoter selon certains paramètres, et ensuite fonctionner de manière autonome grâce à la méthode `track`. Puis, au moment désiré, de lui demander d'éteindre la lumière sans difficulté ou autre gestion supplémentaire.

Généralisation du concept

Il existe une vaste gamme de dispositifs intermittents. Par exemple, il pourrait y avoir :

- une diode électroluminescente (**DEL**) qui ne peut être qu'ouverte ou éteinte
- une **DEL** dont l'intensité lumineuse peut être régulée
- une **DEL** dont la couleur peut être déterminée
- une serrure électrique
- un système de chauffage
- une pompe électrique
- une alarme sonore piézoélectrique (*buzzer*)
- un relais électrique, un électroaimant , une valve, une pompe ou un moteur activant des actions physiques

- ...

Encore une fois, il a été identifié un besoin de généralisation utile tant pour ce projet que pour d'autres. On présente donc ici le concept générique de **Blinker**, un dispositif intermittent à deux états, qui présente les caractéristiques suivantes :

- la classe **Blinker** est une machine d'états
- elle sert à gérer n'importe quel dispositif à deux états (certainement l'un des concepts les plus significatifs pour cette généralisation)
- elle propose des contrôles variés :
 - des actions instantanées comme éteindre ou allumer le dispositif
 - des actions temporisées comme le fait de l'allumer et, qu'après un certain temps, il s'éteigne automatiquement
 - mais aussi des actions récurrentes comme faire clignoter et, optionnellement, l'ajout de patrons comportementaux spécifiques
- elle permet de connaître l'état courant du dispositif en tout temps
- elle permet de changer l'action courante en tout temps
- ne possède pas d'état terminal : le dispositif est toujours *en vie*, en autant que la méthode **track** soit appelée

Conception proposée

La conception repose sur une classe héritant de **StateMachineDevice** offrant une interface de contrôle standardisée pour contrôler un dispositif à deux états : ouvert ou fermé. Le concept est comparable à celui d'un interrupteur ou d'une bascule, dont le contrôle peut être manuel ou automatique. Ainsi, il est possible d'éteindre, allumer ou faire clignoter le dispositif.

Cette classe est spécialisée par son comportement, mais reste générique sur le matériel à contrôler. L'idée est d'exploiter l'infrastructure de la machine d'états, permettant d'utiliser des états spécialisés selon le matériel et ses opérations spécifiques. Puisque les états sont conçus pour inclure des actions automatiques (**_do_entering...**, **_do_in_state...** et **_do_exiting...**), il n'est pas nécessaire que la classe **BlinkerDevice** définisse un autre mécanisme d'extension mais plutôt exploiter le mécanisme déjà en place. L'approche utilisée consiste donc à définir ces états spécialisés lors de l'initialisation de la classe.

La machine d'états est aussi particulière car elle possède non seulement un état initial mais aussi la possibilité d'être réinitialisée dans divers autres états et à n'importe quel moment. Cette approche est intéressante car elle encapsule une multitude de comportements dans la même classe et, indirectement, d'agencer plusieurs machines d'états simplistes dans une « méta machine d'états ». Les unes pouvant communiquer avec les autres. La conception proposée exploite cette possibilité à un niveau introductif mais suffisant pour voir toute la puissance possible et comment ce concept pourrait être encore généralisé.

Enfin, il est essentiel de noter que la machine d'états conserve une dynamique partielle. Le *layout* des états et des transitions est fixe après l'initialisation, lui n'est pas dynamique, mais les paramètres de ces états et transitions sont reconfigurés lors des réinitialisations. Par exemple, l'appel de la méthode **blink** configure certains états et transitions par les arguments donnés via les paramètres **cycle_duration**, **percent_on**, et **begin_on**. Cela nécessite une attention particulière dans la conception de la classe et de ses mécanismes de réinitialisation.

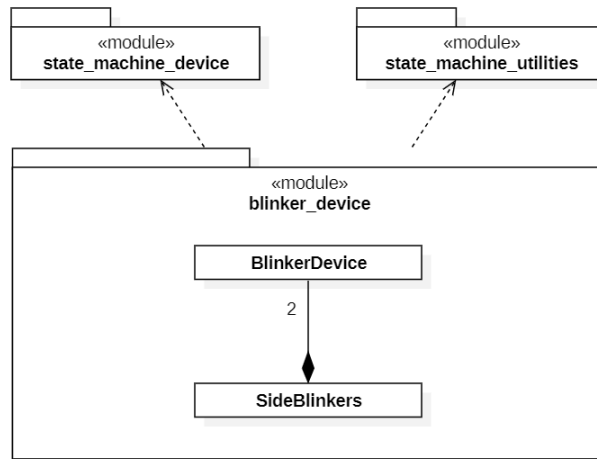
Module **blinker_device**

Le module **blinker_device** offre :

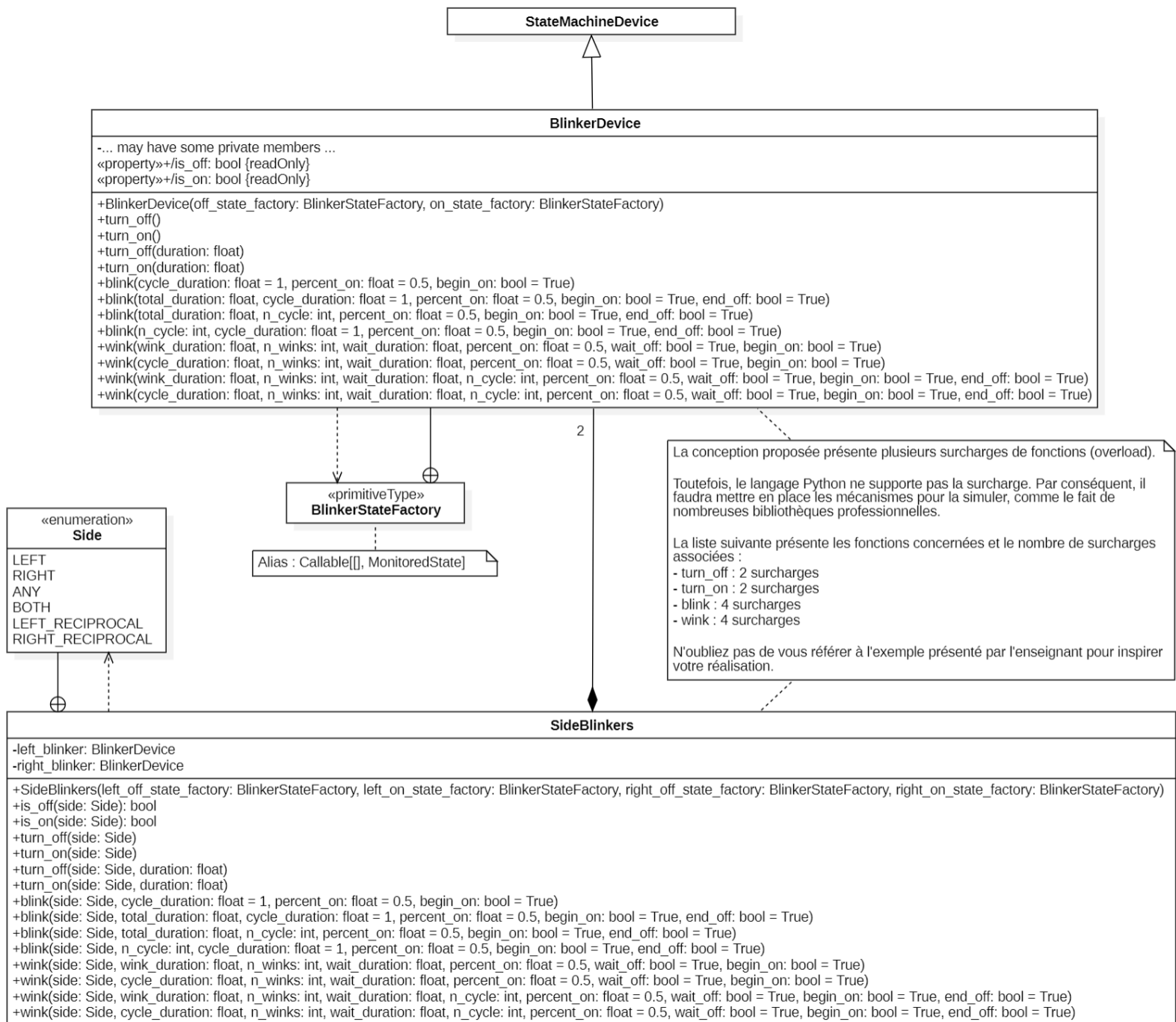
- **BlinkerDevice** : une classe générique offrant la gestion d'un dispositif intermittent
- **SideBlinkers** : une classe générique encapsulant deux **BlinkerDevice** disposés l'un à côté de l'autre.

SideBlinkers est une classe utilitaire servant de facilitateur pour effectuer la tâche de répartition et de synchronisation pour deux **BlinkerDevice** conceptuellement reliés.

Diagramme de classes



Vue d'ensemble du module **blinker_device**



Classes BlinkerDevice et SidesBlinker

Diagramme d'états-transitions représentant le comportement de la classe **BlinkerDevice**

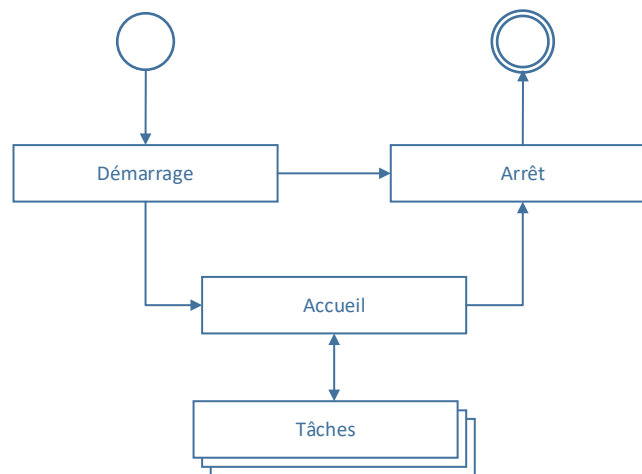
- Vous devez créer un petit programme utilisant un `BlinkerDevice` affichant à l'écran le texte `On` ou `Off` aux changements d'état. Ce projet doit être dans le fichier `project_blinker.py`.

Présentation générale

Enfin, le projet prend forme. Il est temps de développer l'application principale basée sur tous les outils développés.

Tel que présenté au début du document, l'application principale consiste à offrir à l'utilisateur la possibilité de contrôler le robot dans un contexte opérationnel flexible et évolutifs supportant divers types de tâche. Comme nous l'avons vu, l'application est divisée en quatre parties principales :

- Le démarrage : amorce l'initialisation du matériel et s'assure que tout soit intègre
- L'arrêt : assure que le robot soit dans une situation sécuritaire avant l'arrêt définitif du projet
- L'accueil : le logiciel attend l'instruction de l'utilisateur pour engager une tâche spécifique
- L'opération de tâche : une tâche est en cours d'exécution



Sections principales de l'application à développer

Cette division logique permet de bien segmenter les constituants du logiciel et de mieux cerner les objectifs importants :

- Démarrage :
 - Le matériel est en démarrage et on amorce les sous-systèmes du logiciel.
 - Est-ce que l'instance du robot est réalisée sans problème et est disponible?
 - Est-ce que le robot est intègre? C'est-à-dire, est-ce que les capteurs et actionneurs sont accessibles et fonctionnels tels qu'attendu?
- Arrêt :
 - Avant de terminer l'application, le robot doit se trouver dans une situation sécuritaire les utilisateurs et son intégrité physique.
- Accueil :
 - L'application est en attente d'une instruction de l'utilisateur.

- Cet état ne fait rien de spécifique. Le robot est à l'arrêt et un signal visuel évident informe l'utilisateur de cet état.
- Une tâche peut être engagée à n'importe quel moment grâce à l'appui d'une touche sur la télécommande. Les touches numériques 1, 2, 3, ..., 9 sont réservées pour l'activation des tâches.
- À n'importe quel moment il est possible de quitter l'application par l'appui de la touche OK sur la télécommande.
- Tâches :
 - Une tâche est en cours.
 - Les tâches sont génériques dans le sens qu'elles ne sont pas spécifiquement définies dans la couche de l'application.
 - Il existe néanmoins une tâche par défaut, la tâche de contrôle manuel. Cette dernière consiste à pouvoir contrôler directement les mouvements du robot par la télécommande.
 - À n'importe quel moment il est possible de quitter la tâche par l'appui de la touche OK sur la télécommande.
- Dans tous les cas :
 - Une rétroaction pertinente est donnée à l'utilisateur.
 - La rétroaction peut être par :
 - Un message à la console
 - Certains motifs d'affichage sur les phares, les yeux ou la DEL arrière.
 - Autres pourraient être possibles.

Diagramme d'états-transitions

Le diagramme UML d'états-transitions suivant présente l'architecture de l'application. Nous y retrouvons une structure d'états et de transitions permettant le déroulement logique des 4 portions logiciel. On remarque que les trois divisions déjà présentées sont elles-mêmes constituées de sous structures adaptées au contexte.

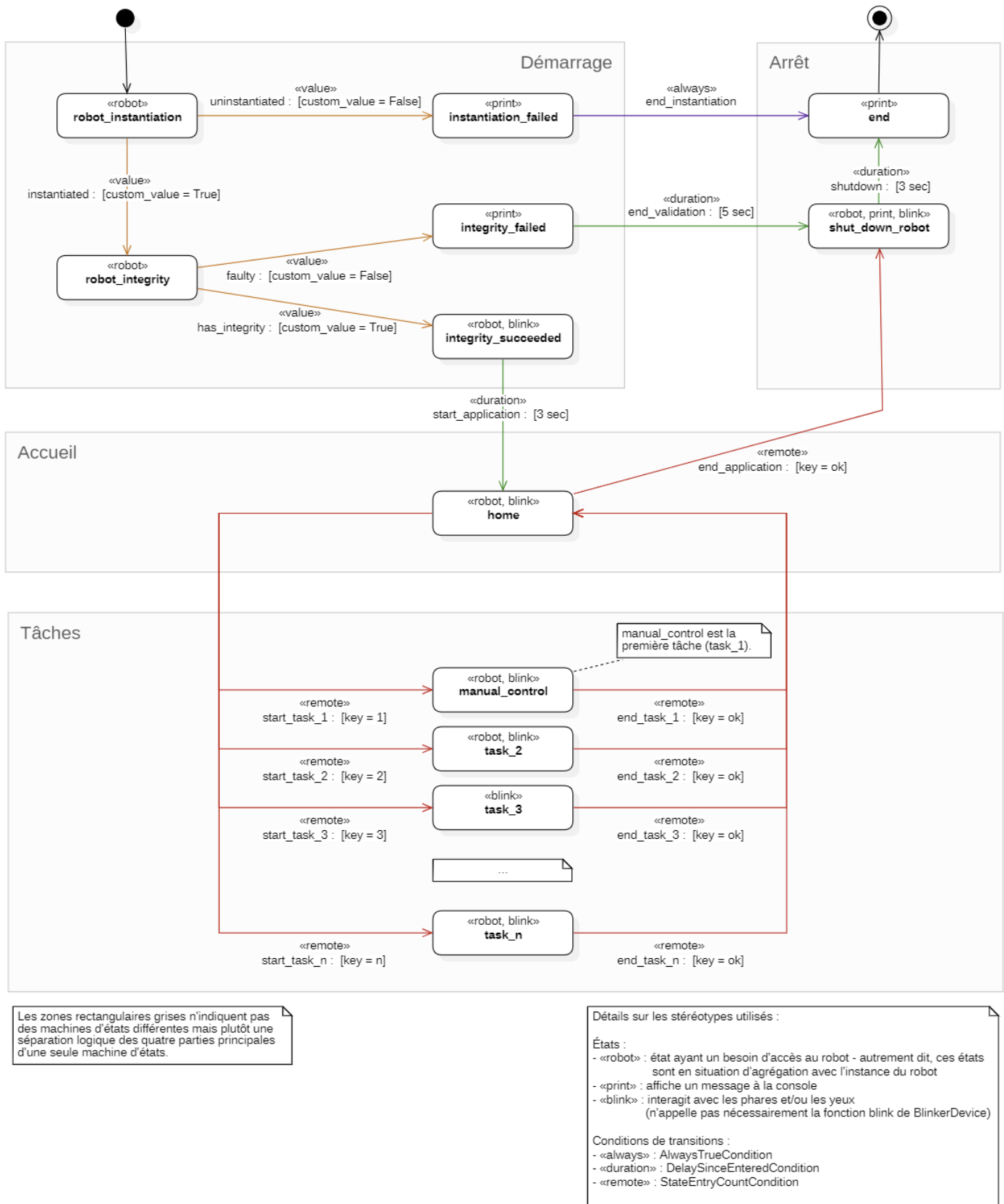


Diagramme d'états de l'application principale

Détails sur chacun des états de l'application principale

Le diagramme d'état précédent présente tous les états de l'application. Chacun de ces états existe pour une raison bien précise et doivent réaliser certaines actions. Voici le détail de ces informations :

- **robot_instantiation**
 - rôle : on s'assure que l'objet représentant le robot soit instancié et disponible
 - opération robot : rien
 - impression : rien
 - phares : rien
 - yeux : rien
 - télécommande : rien
- **instantiation_failed**
 - rôle : représente le fait que le robot ne soit pas accessible et que l'application doit terminer immédiatement
 - opération robot : rien
 - impression : affiche dans la console un message d'erreur approprié
 - phares : rien
 - yeux : rien
 - télécommande : rien
- **end**
 - rôle : le programme est terminé, cet état est terminal.
 - opération robot : rien
 - impression : affiche dans la console un message approprié
 - phares : rien
 - yeux : rien
 - télécommande : rien
- **robot_integrity**
 - rôle : sachant que le robot est disponible, on effectue la validation de l'intégrité du robot, c'est-à-dire qu'on s'assure que le robot fonctionne bien (que tous ses capteurs et actionneurs soient dans un état fonctionnel)
 - opération robot : rien
 - impression : rien
 - phares : rien
 - yeux : rien
 - télécommande : rien
- **integrity_failed**
 - rôle : le robot n'est pas intègre, on doit quitter l'application immédiatement
 - opération robot : rien
 - impression : affiche dans la console un message d'erreur approprié
 - phares : clignote rouge avec un cycle de 0.25 seconde à 50% ouverts et 50% fermés
 - yeux : clignote rouge avec un cycle de 0.25 seconde à 50% ouverts et 50% fermés
 - télécommande : rien
- **shut_down_robot**
 - rôle : amorce de la fin du programme, on s'assure de laisser le robot dans un état valide et sécuritaire

- opération robot : s'il y a des opérations à réaliser avec le robot avant de quitter, c'est ici qu'elles sont effectuées – l'objectif est de s'assurer de la sécurité des utilisateurs et de l'intégrité physique du robot
- impression : affiche dans la console ces messages :
 - d'abord : `shutdown in process...`
 - finalement, sur la même ligne : `... shutdown successful!`
- phares : clignote avec un cycle de 0.5 seconde à 25% ouverts et 75% fermés
- yeux : clignote jaune avec un cycle de 0.5 seconde à 75% ouverts et 25% fermés
- télécommande : rien
- **integrity_succeeded**
 - rôle : le robot est intègre, l'application poursuit son déroulement normal
 - impression : affiche dans la console un message de validation approprié (instanciation et intégrité)
 - phares : rien
 - yeux : clignote vert avec un cycle de 0.5 seconde à 50% ouverts et 50% fermés
 - télécommande : rien
- **home**
 - rôle : l'application est en attente d'une instruction de l'utilisateur pour l'amorce d'une tâche
 - opération robot : rien
 - impression :
 - affiche un message d'actualisation pertinent dans la console
 - le message doit toujours être sur la même ligne
 - le contenu du message est laissé à votre discrétion mais sa pertinence est laissé à votre discrétion
 - on désire néanmoins voir le temps écoulé depuis le démarrage du programme (en minutes, secondes, millisecondes)
 - phares : rien
 - yeux : clignote jaune en alternance avec un cycle de 1.5 seconde à 50% ouverts et 50% fermés
 - télécommande :
 - la touche **OK** quitte l'application
 - les touches numériques de 0 à 9 démarrent la tâche associée
- **task_n**
 - rôle : une tâche est en cours, cette tâche est générique du point de vue de l'application
 - opération robot : selon la tâche
 - impression : selon la tâche
 - phares : selon la tâche, néanmoins on encourage l'utilisation des phares comme rétroaction secondaire à l'utilisateur (par exemple, instruction d'un état interne ou de débogage)
 - yeux :
 - l'oeil gauche clignote à un cycle de 1.0 seconde à 15% ouverts et 85% fermés
 - la couleur de l'œil gauche est propre à la tâche et suit le patron suivant :
 - la couleur est défini dans l'espace colorimétrique **HSL** (teinte-saturation-luminosité)
 - la teinte est défini par le ratio suivant : $hue = \frac{2(task-1)}{3(n-1)}$, où *hue* est la teinte, *task* est le numéro de la tâche (de 1 à *n*) et *n* est le nombre de tâches existantes dans l'application.

- la saturation est au maximum
- la luminosité est à 50%
- l'œil droit **doit** être utilisé mais est propre à la tâche
- télécommande :
 - la touche **OK** quitte la tâche
 - toutes les autres touches sont disponibles pour des actions internes à la tâche

Importance de la structure flexible associée aux tâches

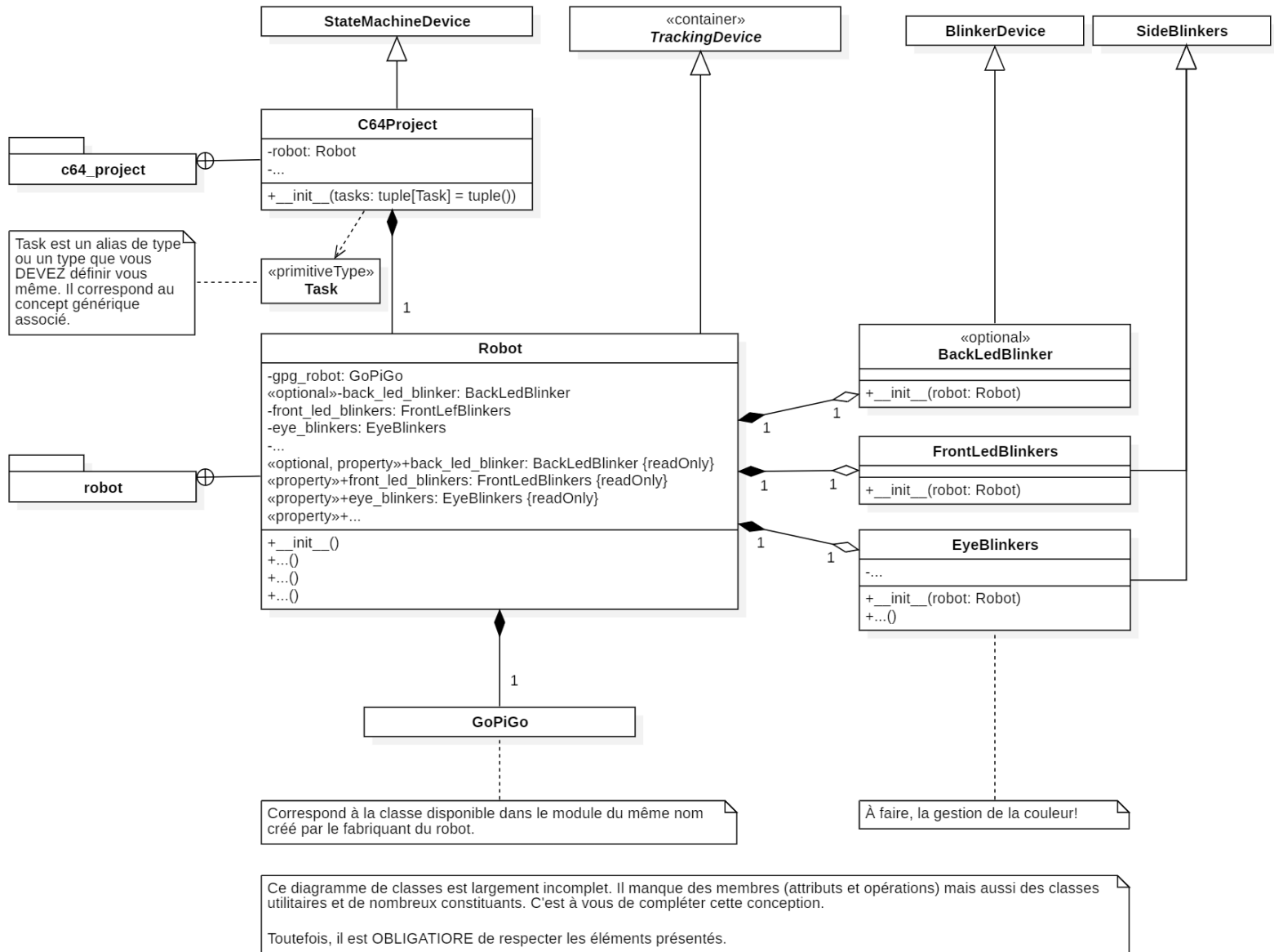
Un concept important du logiciel est sa flexibilité à supporter plusieurs tâches différentes. Ces tâches sont conceptuellement génériques et servent à offrir des mécanismes d'action différents à l'utilisateur. L'approche logicielle est si flexible qu'il est possible d'ajouter des tâches pratiquement aussi simplement qu'un système similaire à un plugiciel (*plugin*). Par défaut, l'application ne possède qu'une seule tâche, le contrôle manuel du robot. Toutefois, il est possible de développer d'autres tâches et de les ajouter au logiciel sans difficulté. D'ailleurs, la dernière étape du projet consiste à développer une telle tâche.

Conception à réaliser

Rendu à cette partie du projet, on ne vous impose plus de structure logicielle stricte (à quelques exceptions). C'est à vous de réaliser votre conception et le développement associé. Toutefois, il est attendu que vous ayez un niveau de qualité logiciel adéquat qui respectent ces contraintes :

- Vous devez utiliser le développement réalisé en respect de sa philosophie tout en exploitant leurs avantages et en connaissant leurs limitations.
- Vous ne pouvez pas modifier les classes des bibliothèques développées pour un aspect spécifiques de cette application.
- Vous devez favoriser autant que possible le paradigme orienté objet lorsqu'il est pertinent et exploité le paradigme fonctionnel lorsqu'il est adapté. Autrement dit, on n'utilise pas un paradigme seulement parce qu'il requiert moins de ligne de code.
- Vous devez respecter minimalement les indices qui vous sont donnée dans les diagrammes UML donné plus loin. C'est-à-dire que vous devez produire les classes **C64Project** et **Robot** en respectant les fonctions d'initialisation `__init__` et les associations indiquées (héritage et composition).
- Vous devez développer une classe faisant l'encapsulation du robot (**EasyGoPiGo3**). Cette classe est l'une des parties les plus importantes du projet.
- ...

Diagrammes UML



Diagrammes des modules `c64_project` et `robot` ainsi que des classes `C64Project`, `Robot` et quelques classes utilitaires

Tâche de contrôle manuel

Votre projet doit posséder systématiquement une tâche par défaut, la tâche `manual_control`. Cette tâche sert à la fois :

- Offrir à l'utilisateur une tâche simple pouvant effectuer le déplacement du robot.
- C'est à la fois une tâche de débogage du point de vue de l'utilisateur pour voir si les composants suivants fonctionnent bien :
 - phare
 - yeux
 - télécommande (du moins pour les touches `ok`, `up`, `down`, `left` et `right`)
 - mouvement (moteurs)
- De point de vue du développeur de l'application, c'est une opportunité de débogage pour l'intégration de tâche dans l'application.
- Du point de vue d'un développeur de tâches, c'est l'équivalent d'un *Hello World!*. Le développement de cette tâche permet de réaliser une simple tâche et de l'intégrer dans l'application. Cet effort permet de passer par toutes les étapes d'intégration incluant les débogages nécessaires.

Détails sur l'état `manual_control`

- `manual_control`
 - rôle : contrôle manuel des déplacements du robot
 - impression : rien
 - mouvement du robot : selon les touches prescrites (voir plus bas) le robot effectue le déplacement associé
 - phares :
 - robot à l'arrêt : phares éteints
 - robot avance : les phares clignotent avec un cycle de 0.5 seconde à 75% ouverts et 25% fermés
 - robot recule : les phares clignotent avec un cycle de 0.5 seconde à 25% ouverts et 75% fermés
 - robot tourne à droite : le phare gauche est éteint alors que le phare droit clignote avec un cycle de 0.5 seconde à 50% ouvert et 50% fermé
 - robot tourne à gauche : le phare droit est éteint alors que le phare gauche clignote avec un cycle de 0.5 seconde à 50% ouvert et 50% fermé
 - yeux :
 - l'œil gauche selon le patron prescrit
 - l'œil droit :
 - si le robot est à l'arrêt clignote magenta avec un cycle de 1.5 seconde à 15% ouverts et 85% fermés
 - si le robot est en déplacement clignote magenta avec un cycle de 0.5 seconde à 50% ouverts et 50% fermés
 - télécommande :
 - la touche `OK` quitte la tâche
 - si aucune touche n'est appuyée, le robot est à l'arrêt
 - si la touche `up` est appuyée, le robot avance
 - si la touche `down` est appuyée, le robot recule

- si la touche **left** est appuyée, le robot tourne à gauche
- si la touche **right** est appuyée, le robot tourne à droite

Diagramme états-transitions

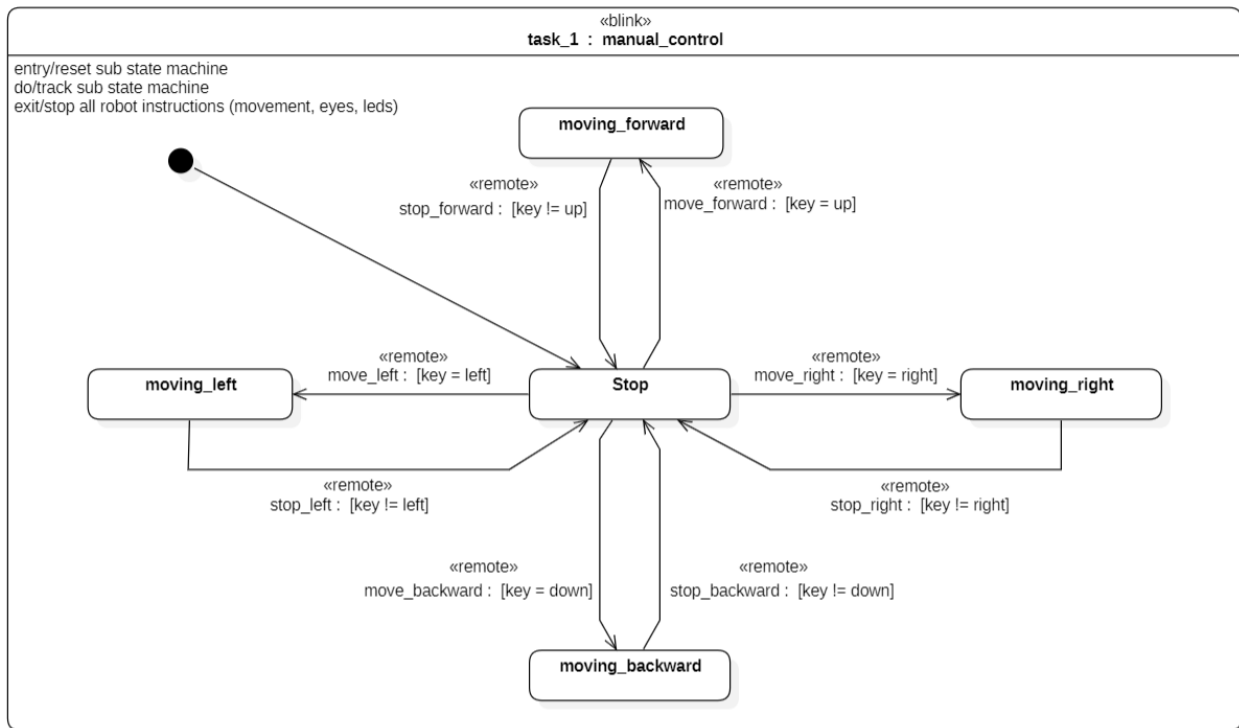


Diagramme d'états de la tâche `manual_control`

Détails sur les états de la tâche `manual_control`

Voici les détails des états de cette tâche :

- **stop**
 - rôle : le robot est à l'arrêt
 - opération robot : rien
 - impression : rien
 - phares : rien
 - yeux :
 - l'œil gauche selon le patron prescrit
 - l'œil droit clignot magenta avec un cycle de 1.5 seconde à 15% ouverts et 85% fermés
 - télécommande : les touches **up**, **down**, **left** et **right** engage les états associés
- **moving_forward**
 - rôle : le robot avance
 - opération robot : rien
 - impression : rien

- phares : les phares clignotent avec un cycle de 0.5 seconde à 75% ouverts et 25% fermés
- yeux :
 - l'œil gauche selon le patron prescrit
 - l'œil droit clignote magenta avec un cycle de 0.5 seconde à 50% ouverts et 50% fermés
- télécommande : le relâchement de la touche **up** engage le retour à l'état **stop**
- **moving_backward**
 - rôle : le robot recule
 - opération robot : rien
 - impression : rien
 - phares : les phares clignotent avec un cycle de 0.5 seconde à 25% ouverts et 75% fermés
 - yeux :
 - l'œil gauche selon le patron prescrit
 - l'œil droit clignote magenta avec un cycle de 0.5 seconde à 50% ouverts et 50% fermés
 - télécommande : le relâchement de la touche **down** engage le retour à l'état **stop**
- **moving_left**
 - rôle : le robot tourne à gauche
 - opération robot : rien
 - impression : rien
 - phares : le phare droit est éteint alors que le phare gauche clignote avec un cycle de 0.5 seconde à 50% ouvert et 50% fermé
 - yeux :
 - l'œil gauche selon le patron prescrit
 - l'œil droit clignote magenta avec un cycle de 0.5 seconde à 50% ouverts et 50% fermés
 - télécommande : le relâchement de la touche **left** engage le retour à l'état **stop**
- **moving_right**
 - rôle : le robot tourne à droite
 - opération robot : rien
 - impression : rien
 - phares : le phare gauche est éteint alors que le phare droit clignote avec un cycle de 0.5 seconde à 50% ouvert et 50% fermé
 - yeux :
 - l'œil gauche selon le patron prescrit
 - l'œil droit clignote magenta avec un cycle de 0.5 seconde à 50% ouverts et 50% fermés
 - télécommande : le relâchement de la touche **right** engage le retour à l'état **stop**
- À n'importe quel moment, l'activation de la touche **ok** quitte la tâche. Attention, il est important de comprendre cette partie du programme car son impact est fondamental et déterminant.

Développement attendu

Vous devez développer les classes suivantes dans les modules spécifiés :

- **C64Project**
- **Robot** (incluant **FrontLedBlinkers** et **EyeBlinker**, optionnellement **BackLedBlinkers**)

- La ou les classes nécessaires pour réaliser la tâche 1, `manual_control` (non détaillées dans les diagrammes – ceci est un indice, à vous de faire la meilleure conception réutilisable autour de la notion de `Task`).
- Toutes autres classes et outils nécessaires à votre implémentation.

Livrables attendus

- Toutes les classes demandées et proposées sont à mettre dans les modules appropriés.
- Il est attendu d’avoir une conception orienté objet de haut niveau.
- Il est attendu que le projet fonctionne tel qu’attendu.

Sachez qu’une attention particulière doit être faite à la qualité de la conception et du développement.

Présentation générale

Les parties antérieures du projet consistaient à mettre en place une infrastructure puissante, flexible et réutilisable. Cette dernière partie consiste à étendre les capacités de l'application en ajoutant une tâche supplémentaire qui permet d'exploiter les capacités du robot.

Sujet de la tâche

Le sujet de cette nouvelle tâche est ouvert et c'est à chaque équipe de trouver quels aspects elle désire explorer et exploiter.

Contraintes techniques

Votre nouvelle tâche doit répondre minimalement à ces contraintes de réalisation :

- Votre tâche doit s'ajouter au logiciel existant en respectant la philosophie de l'infrastructure développée.
- Votre tâche est un état dans le logiciel, au même titre que l'état `manual_control` :
 - Lorsque l'application se trouve dans l'état accueil (`home`), la touche 2 de la télécommande engage la transition de l'état d'accueil à l'état correspondant à votre nouvelle tâche.
 - Lorsque l'application se trouve dans votre nouvel état, vous pouvez la quitter :
 - À n'importe quel moment (ou presque) avec l'appui de la touche `ok` sur la télécommande.
 - Optionnellement, vous pouvez faire en sorte que la tâche quitte par elle-même selon une logique spécifique pertinente pour cette tâche.
 - Dans les deux cas, l'application retourne à l'état d'accueil sans quitter l'application principale.
- La tâche doit respecter ces considérations et contraintes :
 - La tâche doit être intéressante et avoir un objectif observable.
 - Le robot doit bouger.
 - En tout temps, l'oeil gauche doit clignoter selon le patron prescrit alors que l'oeil droit doit représenter le déroulement en cours. Le rythme de clignotement, le pourcentage ouvert/fermé ou la façon de faire clignoter est libre.
 - Les phares du robot doivent être utilisés pour donner de la rétroaction à l'utilisateur (au moins une, ne fût-ce qu'une information de débogage).
 - Le robot doit utiliser les deux capteurs suivants :
 - télécommande (en plus de la touche `ok` pour quitter la tâche)
 - télémètre
 - Au moins un servo doit être utilisé. ATTENTION – IMPORTANT : vous devez obligatoirement limiter la rotation des servomoteurs par programmation en tout temps – une encapsulation de qualité est attendue :
 - 45° à 135° pour la caméra sur le port SERVO1 configurable selon le nouveau référentiel $90^\circ \pm 45^\circ$
 - 35° à 145° pour le télémètre sur le port SERVO2 configurable selon le nouveau référentiel $90^\circ \pm 55^\circ$

Qualité de la tâche

Le sujet de cette tâche est ouvert dans sa forme et vous donne une certaine liberté de réalisation. Toutefois, sachez que l'évaluation finale tiendra compte des contraintes mentionnées ainsi que des critères suivants :

- Originalité de votre tâche :
 - Originalité
 - Pertinence dans le contexte de l'utilisation d'un robot mobile
 - Niveau de difficulté
- Vous devez faire un effort pour réaliser votre projet de façon à :
 - valoriser ce que le robot peut faire
 - utiliser intelligemment l'infrastructure créée dans la première partie et insérer cette nouvelle tâche sans complexité du côté de l'application principale
 - démontrer vos compétences techniques en réalisant les abstractions possibles lorsqu'elles se présentent.

Finalisation du projet

Rapport

Vous devez produire un petit rapport en format **Markdown** dans un fichier nommé **c64_rapport.md** situé à la racine du projet répondant à ces questions :

- Qui sont les membres de l'équipe?
- Concernant chacune des quatre premières phases de développement :
 - donnez, en pourcentage, un estimé de la proximité de ce que vous avez réalisé et la conception donnée – autrement dit, à quel point votre travail réalise ce qui a été demandé
 - si votre pourcentage est inférieur à 100%, nommez quels sont les éléments non terminés et, succinctement, expliquez pourquoi
- Donnez une courte description de l'infrastructure réalisée pour les éléments suivants (on cherche à comprendre quelles sont les abstractions réalisées) :
 - classe représentant le robot :
 - validations de l'instanciation et de l'intégrité du robot
 - gestion de la télécommande
 - gestion du télémètre et de son servo moteur
 - gestion des moteurs
 - gestion de la couleur pour les yeux (la classe **EyeBlinkers**)
 - structure générale du logiciel :
 - quelle est l'infrastructure générale du logiciel
 - capacité modulaire d'insertion d'une nouvelle tâche
 - Quels sont les patrons de conception utilisés à travers ce projet. On cherche autant les patrons de conception évidents et bien définis que ceux plus subtils et utilisant des implémentations moins formelles. Autant ceux existants dans la conception imposée que ceux que vous ajoutez.
 - Expliquez quel est l'impact de l'utilisation du patron de conception machine d'état sur le développement du projet. On cherche à comprendre les avantages et les inconvénients de ce patron de conception. Discutez ensuite des différences et de l'impact si nous avions décidé de rien utiliser pour réaliser les mêmes fonctionnalités.
 - Présentez la conception que vous avez réalisée pour la réalisation de la phase 5. On cherche à bien comprendre les valorisations que vous avez faites autour de ce qui est demandé, des outils existants et comment vous avez exploité tout ça.
 - Y a-t-il d'autres éléments d'abstraction sur lesquels que vous aimeriez porter une attention particulière?
- Pour la phase 6 de développement, votre tâche personnalisée :
 - Donnez une description de la tâche que vous avez implémentée.
 - Rôle de la tâche.
 - Description technique de :
 - Que font les yeux?
 - Que font les phares?
 - Quel déplacement le robot fait-il?
 - Quels sont les objectifs des capteurs (télécommande et télémètre)?

- Quel servo utilisez-vous et à quel dessein? Quelle est votre stratégie de gestion pour la contrainte de limitation des angles?
 - Quelles sont les abstractions que vous aimeriez porter à l'attention du correcteur?
- Globalement, considérant l'ensemble du projet :
 - Identifiez et expliquez sommairement quelles ont été les parties les plus difficiles dans le projet en considérant :
 - l'ensemble des phases 1 à 4 (pas chacune d'entre elles)
 - la phase 5
 - la phase 6
 - Donnez votre impression sur la pertinence:
 - d'exploiter le paradigme orienté objet
 - d'utiliser le langage Python (pensez-vous que ce projet aurait été possible avec les langages Java, C# et C++?)
 - d'utiliser les annotations de type et l'utilisation de `mypy`
 - d'utiliser la `docstring`
 - d'utiliser les tests unitaires et les `doctest`
 - de baser la conception sur des patrons de conception
 - de faire un projet ayant une certaine envergure avec une aussi grosse équipe
- Individuellement, un court paragraphe par membre de l'équipe : qu'avez-vous trouvé le plus formateur dans ce projet?

Sachez qu'il est attendu que vous soyez extrêmement concis et précis pour ce rapport. On ne désire aucun texte philosophique ou de remplissage. On cherche un texte très technique présenté sous forme de puces pertinentes. Vous devez utiliser un vocabulaire hautement technique. Visez l'équivalent d'un maximum de 5 pages.

Stratégie d'évaluation

L'évaluation se fera en 2 parties. D'abord, l'enseignant évaluera le projet remis et assignera une note de groupe pour le travail. Ensuite, chaque équipe devra remettre un fichier Excel dans lequel sera soigneusement reportée une cote représentant la participation de chaque étudiant dans la réalisation du projet. Cette évaluation est faite en équipe et un consensus doit être trouvé.

Une pondération appliquée sur ces deux évaluations permettra d'assigner les notes finales individuelles.

Ce projet est long et difficile. Il est conçu pour être réalisé en équipe. L'objectif est que chacun prenne sa place et que chacun laisse de la place aux autres.

Ainsi, trois critères sont évalués :

- participation (présence en classe, participation active, laisse participer les autres, pas toujours en train d'être sur Facebook ou sur son téléphone, concentré sur le projet, pas en train de faire des travaux pour d'autres cours ...)
- réalisation (répartition du travail réalisé : conception, modélisation, rédaction de script, documentation ...)
- impact (débrouillardise, initiative, amène des solutions pertinentes, motivation d'équipe ...)

Remise

La remise est implicite via `GitHub`. Mettre un `tag` final nommé `remise` lorsque vous avez terminé le projet.

On vous demande de laisser votre projet privé sur GitHub.

Annexe 1 – UML

Rappel sur la notation UML et directives supplémentaires générales

Rappel sur la notation UML

- classe : un carré divisé verticalement en trois
- masquage :
 - privé
 - # protégé
 - + publique
- les types sont à droite séparés par un deux-points :
- les associations sont :
 - héritage : flèche blanche
 - dépendance : flèche pointillée
 - composition : losange noir
 - agrégation : losange blanc
 - contenu : petit cercle blanc avec un + au centre
spécifie que l'élément lié est un sous-élément
- concept d'élément abstrait, en italique
- concept de membre statique, souligné
- concept de membre dérivé, barre oblique /
- concept de paquetage (package), un rectangle avec onglet en haut à gauche
- concept de module est représenté par un package ayant le stéréotype «module»
- concept de généralité de type, un petit encadré pointillé dans le coin supérieur droit avec le nom du paramètre (dans les exemples ici, T)
(communément appelé generic ou template selon les langages)
- le concept de lecture seule pour une variable, la propriété { readOnly }
- le concept de lecture seule pour une fonction, la propriété { query }
- le concept d'amitié, le stéréotype «friend»
- note
 - une note (un commentaire) un rectangle avec le coin supérieur droit replié
 - une ligne pointillée indique à quoi la note fait référence

Toutes les classes importantes doivent implémenter, explicitement ou implicitement, les 'dunder methods' suivantes :

- __str__
- __repr__.

Par soucis de concision, cette information n'est pas répétée dans chacune des classes.

Le choix de créer ces fonctions repose entre vos mains. Il est important de comprendre que pour chaque classe, vous devriez réfléchir sur le fait de le faire ou non.

Signification des stéréotypes utilisés

Notation standard UML :

- «primitiveType» : alias de type
- «enumeration» : énumération (Enum)

Notation liée au langage Python ou à certains concepts POO :

- «module» : module Python (fichier *.py)
- «property» : propriété
(s'assurer de comprendre les notions lecture-écriture ou lecture seule)
- «override» : substitution de fonction polymorphisme
- «final» : substitution de fonction polymorphique finale
(à ne pas confondre avec l'annotation de type Final)
- «friend» : la notion d'amitié

Notation propre aux diagrammes d'états :

- «always» : condition toujours effective
- «value» : fait référence à une condition basée sur la valeur de l'état en surveillance
- «duration» : fait référence à une condition basée sur la durée écoulée depuis l'activation de l'état en surveillance
- «count» : fait référence à une condition basée sur le nombre de fois que l'état a été en surveillance a été effectif
- «remote» : fait référence à une transition basée sur la valeur de la télécommande du robot

Autres notations

- «container» : classe encapsulant des données
(voir la note au bas)
- «optional» : la réalisation est optionnelle, n'est présent qu'à titre d'exemple supplémentaire ou d'amélioration possible

Le stéréotype <<container>> indique une classe possédant une structure de données interne qui pourraient être exposée via l'interface de programmation publique. Toutefois, on désire ne pas exposer directement la structure de données publiquement.

Ainsi, selon le cas, il est possible, voir encouragé, d'implémenter certaines 'dunder method' parmi les suivantes :

- __getitem__
- __setitem__
- __delitem__
- __iter__
- __reversed__
- __contains__
- __len__
- __bool__

Vous devez considérer cette information comme une opportunité d'augmenter les interfaces de programmation des classes concernées.

Rappels et précisions sur la notation UML